VideoGenDoctor: Structured Diagnosis, Evidence Localization, and Repair for Controllable Video Generation

Bo Tan∗
College of Information Engineering
Sichuan Agricultural University
Ya'an, Sichuan 625014, China
bot@stu.sicau.edu.cn

Yupeng Niu
Tianjin Key Laboratory of Radiation Medicine and Molecular Nuclear Medicine
Institute of Radiation Medicine, CAMS & PUMC
Tianjin 300192, China
niuyupeng1@gmail.com

Abstract

Controllable video generation pipelines now condition on character identities, camera trajectories, required props, and structured shot scripts. When the output violates these specifications, scalar quality scores from current evaluators cannot localize the failure or recommend a repair. VideoGenDoctor replaces the scalar score with structured diagnosis records. Given a video and a specification, it outputs failure-as-code tuples that each carry a taxonomy code, temporal evidence span, affected target, confidence, representative keyframes, and a compiled repair action.

The default pipeline combines lightweight feature-based screening (CLIP embedding drift, optical flow, face embedding drift, object detection), optional structured VLM verification, and a template-driven patch compiler. On VideoGenDoctor-Bench-v0—a controlled diagnostic fixture of 324 videos spanning 9 perturbation types and 2,016 human-verified evidence spans across 12 observed taxonomy codes—the default diagnosis configuration reaches macro-F1 0.911 and evidence-localization tIoU@0.5 of 0.732. In closed-loop repair under blind human evaluation, VideoGenDoctor-full lifts Pass@2 from 27.8% (score-only) to 76.4%. On a 240-video failure-enriched subset from four production generators, the pipeline retains macro-F1 0.826 and tIoU@0.5 0.598. These results establish that structured diagnosis records with localized evidence enable actionable repair under controlled perturbations. Open-world generalization beyond the current generator and domain coverage remains unverified.

1 Introduction

Diffusion-based video generators now accept text, reference images, camera trajectories, character identities, and structured shot scripts as conditioning signals Yang et al. [2024], Blattmann et al. [2023], Wang et al. [2023], Yuan et al. [2024]. In a production workflow, a ShotIR-style specification defines per-shot props, characters, camera movement, shot type, and action order. When the output

∗Corresponding author.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

violates these specifications, the user needs fine-grained diagnostics: which failures occurred, where in the timeline, which targets are affected, and what should change in the next generation.

Existing evaluators—VBench Huang et al. [2024], EvalCrafter Liu et al. [2024], VideoScore He et al. [2024], T2V-CompBench Sun et al. [2025]—return scalar quality or alignment scores. These scores rank models but do not localize failures or recommend repairs. A low score offers no signal about whether character identity drifted, a required prop disappeared, camera motion was violated, or a specific segment should be regenerated.

VideoGenDoctor replaces the scalar score with structured diagnosis records. Given a video V and a specification S, it outputs

$$R = \{(c_i, t_0^i, t_1^i, y_i, \sigma_i, K_i, T_i, a_i)\},$$

where c_i is a taxonomy failure code, (t_0^i, t_1^i) is the temporal evidence span, y_i names the affected target, σ_i is a confidence score, K_i provides representative keyframes, T_i stores optional track-level evidence, and a_i is a compiled repair action. Each record is both machine-readable and human-auditable.

Three empirical questions follow from this framing. (1) Do structured diagnosis records improve failure-code detection and temporal localization over direct VLM critiques? (2) Do localized evidence and template-constrained repair plans improve downstream video quality beyond score-only feedback? (3) To what extent do conclusions from deterministic perturbations transfer to naturally occurring generator failures?

Scope. VideoGenDoctor-Bench-v0 is a controlled diagnostic fixture, not a general open-world benchmark. Its 324 deterministic-perturbation videos isolate known failure modes and support auditable comparison of diagnosis interfaces. The 240-video failure-enriched real-generator subset provides an initial transfer check but cannot estimate natural failure prevalence. Coverage across arbitrary generators, domains, styles, and long-form videos is not established and remains future work.

Contributions. This paper introduces: a code–span–target–plan representation for controllable video debugging; a modular diagnosis pipeline that combines feature-based candidate generation, optional structured VLM verification, evidence aggregation, and template-driven patch compilation; VideoGenDoctor-Bench-v0 (324 controlled videos, 2,016 verified evidence spans, 9 perturbation types, 12 observed taxonomy codes, plus a 240-video real-generator subset with 18 observed codes); and an evaluation spanning direct VLM comparisons, blind human repair validation, per-code reliability analysis, paired bootstrap tests, and transfer checks on naturally occurring failures.

2 Related work

Video generation evaluation. VBench Huang et al. [2024], EvalCrafter Liu et al. [2024], VideoScore He et al. [2024], and T2V-CompBench Sun et al. [2025] measure aggregate quality, motion smoothness, subject consistency, and text-video alignment. They rank models effectively but do not localize failures or produce repair plans. VideoGenDoctor replaces the scalar score with temporally grounded, code-labeled records linked to repair templates (Appendix A, Table 12).

VLM critics and temporal grounding. Vision-language models can inspect sampled frames and produce natural-language critiques Liu et al. [2023], Chen et al. [2024], OpenAI [2023]. Direct VLM reporting and VLM-to-patch generation are therefore treated as strong baselines, not strawmen. The key question is whether constraining VLM output through structured code–span–target records improves temporal grounding, schema validity, and downstream repair utility. Temporal alignment is measured by intersection-over-union:

$$\text{tIoU} = \frac{\max\big(0, \min(\hat{t}_1, t_1^*) - \max(\hat{t}_0, t_0^*)\big)}{\max(\hat{t}_1, t_1^*) - \min(\hat{t}_0, t_0^*)}.$$

Automated repair. Program repair localizes defects, classifies them, and generates patches. VideoGenDoctor adapts this pattern to video generation: each diagnosis record exposes evidence and maps to a supported repair family, linking evaluation directly to iterative re-generation.

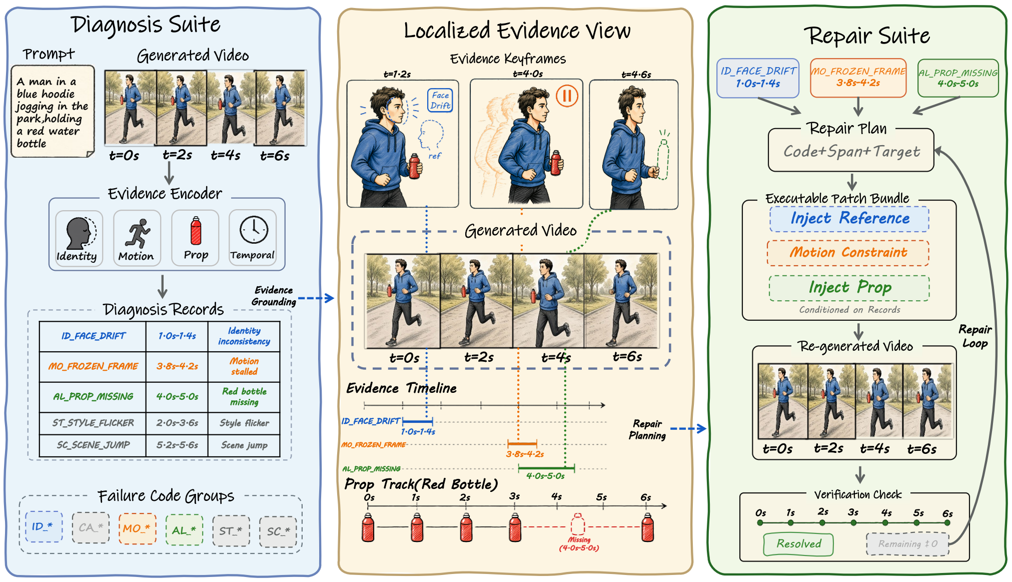

Figure 1: VideoGenDoctor pipeline: segmentation, feature extraction, rule-based candidate diagnosis, optional VLM verification, patch compilation, and closed-loop re-rendering.

3 VideoGenDoctor

3.1 Overview

Figure 1 illustrates the pipeline. It has four stages. (1) The video is partitioned into fixed-stride segments; each segment is sampled into keyframes and four lightweight signals are extracted. (2) Thresholded rules map feature patterns to failure-code candidates with segment-level confidence scores. (3) An optional VLM judge answers structured yes/no questions on top-ranked candidates and produces calibrated confidence scores. (4) A patch compiler maps retained diagnosis records to repair actions: reference-frame injection, prop injection, camera override, segment regeneration, or style correction. A repair plan is *valid* when it names a supported action, target, and evidence span; it is *executable* only when the downstream generator or editor exposes the required control interface.

3.2 Failure records and patch templates

The taxonomy groups 38 failure codes into six categories: Identity, Camera, Motion, Alignment, Style, and Scene. The controlled fixture evaluates 12 observed codes; the real-generator subset covers additional naturally occurring codes. The remaining codes define an extensible namespace and are definitional rather than empirically validated (Appendices A and C). Table 10 in Appendix A lists representative failure-to-signal-to-patch mappings.

Each predicted failure is a tuple

$$r = (c, t_0, t_1, y, \tilde{\sigma}, K, T, a),$$

where c is the taxonomy code, (t_0, t_1) spans the evidence, y names the affected target, $\tilde{\sigma}$ is the confidence, K holds representative keyframes, T stores optional track data, and a is the compiled repair action. The schema is detector- and judge-agnostic.

3.3 Candidate diagnosis and verification

The video is split into 2-second non-overlapping segments; six keyframes are sampled per segment. Four signals are extracted: semantic embedding drift (OpenCLIP Cherti et al. [2023]), optical flow (RAFT Teed and Deng [2020] or Farneback Bradski [2000], Farnebäck [2003]), face embedding drift (InsightFace Deng et al. [2019]), and object/prop cues (YOLOv8 Ultralytics [2023], optional). For each segment s and failure code c, a rule ρ_c maps the feature vector to a score $\sigma_{s,c} \in [0, 1]$. Top-ranked candidates inherit the segment's temporal bounds and the keyframes with highest per-frame anomaly scores.

The optional Stage-2 judge receives the top-K candidates ($K = 3$), their keyframes, temporal bounds, candidate codes, and relevant specification fields. It answers structured yes/no questions rather than generating free-form text. The final confidence blends Stage-1 and Stage-2 scores:

$$\tilde{\sigma}_{s,c} = (1 - \alpha)\sigma_{s,c}^{(1)} + \alpha\sigma_{s,c}^{(2)},$$

with $\alpha = 0.6$. All judge calls log the model, prompt, raw response, latency, and output path.

3.4 Evidence localization and repair planning

Each retained record packs an evidence object $E = (t_0, t_1, K, T)$ for auditability. The patch compiler emits either ShotIR field-level diffs (when a structured specification is available) or generator-agnostic repair plans, preserving the triggering code, evidence span, target, and rationale. In closed-loop mode, a video is considered fixed when all failure confidences drop below $\tau = 0.5$, or after $K_{\max} = 3$ iterations. Because this stopping criterion depends on the system verifier, Section 4.5 reports independent blind human validation as the primary metric.

4 Experiments

4.1 Benchmark and evaluation protocol

VideoGenDoctor is evaluated as a diagnosis-and-repair system, not a video generator. The experiments target the three questions from the introduction: failure-code correctness, evidence localization, and repair usefulness under blind human judgment.

Controlled fixture. VideoGenDoctor-Bench-v0 contains 324 videos built from 18 clean source clips across six domain groups. Each clip is paired with a ShotIR-style specification and perturbed by nine deterministic operators under two random seeds, yielding 2,016 human-verified evidence spans across 12 observed taxonomy codes. The fixture isolates known failure modes to validate interface consistency, evidence auditability, and repair-plan usefulness under controlled conditions. Construction details and operator-to-template alignment are in Appendix A (Tables 14) and Appendix B (Table 20).

Real-failure and real-normal subsets. We collect 240 naturally occurring failure videos from CogVideoX, Wan, Stable Video Diffusion, and HunyuanVideo Yang et al. [2024], Wan-AI [2025], Blattmann et al. [2023], Tencent Hunyuan Foundation Model Team [2025]. From 480 raw generations, videos are retained only when annotators confirm at least one visible specification violation. Sampling is balanced at 60 failures per generator, with generator-specific input adapters logged in per-video manifests: CogVideoX and Wan use ShotIR-to-prompt templates, SVD uses reference-frame conditioning, and HunyuanVideo uses a multi-shot prompt template. An additional 120 real-normal videos (30 per generator) are retained from the same raw pool to estimate false positives. Because the subset is failure-enriched by construction, it supports transfer analysis under verified failures but cannot estimate natural failure prevalence. Source composition, normal-set behavior, and stress tables are in Appendix B.

Annotation. A 120-video reliability subset with 768 candidate evidence spans is dual-annotated before adjudication. Code-level agreement reaches Cohen's $\kappa = 0.831$; span-level agreement is lower (tIoU = 0.681), reflecting inherent ambiguity in fine-grained temporal boundaries. Dataset statistics and annotation details are in Appendix B (Tables 11–26).

4.2 Compared methods and metrics

We compare three groups of methods. **Diagnosis variants:** Prior-only (a frequency-based prior that predicts the most common failure codes per perturbation type without inspecting the video), Rule-only, Rule+DummyJudge, Rule+Open-VLM, Rule+GPT-4V, and VideoGenDoctor-diagnosis.

Table 1: Failure-code detection on the controlled fixture. VideoGenDoctor-diagnosis outperforms direct VLM reporting; Rule+GPT-4V sets the upper reference. Bootstrap 95% CIs in brackets.

Method	Macro-F1	Micro-F1	95% CI
Prior-only	0.4479	0.4635	[0.419, 0.477]
Rule+DummyJudge	0.8584	0.8716	[0.836, 0.879]
Rule-only	0.8926	0.9038	[0.872, 0.911]
Rule+Open-VLM	0.9047	0.9175	[0.885, 0.922]
VideoGenDoctor-diagnosis	0.9110	0.9243	[0.893, 0.928]
Rule+GPT-4V	**0.9276**	**0.9386**	[0.912, 0.941]
VLM free-form report	0.8402	0.8627	[0.814, 0.866]
VLM structured report	0.8869	0.9018	[0.864, 0.908]
Human annotator (adjudicated)	0.958	0.967	[0.942, 0.972]

Repair variants: VideoGenDoctor-patch (applies template-based patches directly from diagnosis output without verification), Patch+Judge, and VideoGenDoctor-full. **External VLM baselines:** free-form VLM reporting, structured VLM reporting, self-consistency, temporal proposal + patch, VLM-to-patch, and Score+LLM-to-patch. Direct VLM baselines share the same GPT-4V endpoint as Rule+GPT-4V to isolate prompt structure, output schema, and repair protocol from model backbone; Rule+Open-VLM provides an open-model reference. Diagnosis-only front ends are paired with the same downstream repair loop whenever human repair success is reported. Implementation details are in Appendix B (Table 27); strengthened VLM-agent baselines are in Appendix A (Table 15).

Metrics include macro-F1 and micro-F1 for failure-code detection; tIoU@0.3/0.5 and Top-K keyframe hit rate for evidence localization; automatic and blind human Pass@1/2 for repair; patch usefulness (5-point Likert); new-artifact rate; and video-level bootstrap 95% confidence intervals. Span-aware metrics use Hungarian matching Kuhn [1955] with tIoU as the primary score and confidence as a tie-breaker. Invalid or missing spans receive no localization credit. Paired comparisons report bootstrap differences; small estimates with confidence intervals crossing zero are treated as inconclusive.

4.3 Failure-code detection

Table 1 reports failure-code detection results. VideoGenDoctor-diagnosis outperforms both free-form and structured direct VLM reporting on macro-F1; Rule+GPT-4V sets the upper reference. The human adjudicated row demonstrates that even expert annotators do not reach perfect F1 on the controlled fixture. Table 2 reports evidence localization results, showing a corresponding improvement in temporal grounding over VLM baselines.

4.4 Per-code reliability and evidence localization

The controlled fixture covers twelve observed codes, with per-code F1 spanning from 0.861 (event-missing) to 0.958 (compression-artifact). Frozen-frame detection reaches the highest F1 (0.956) and tIoU@0.5 (0.794), benefiting from an unambiguous near-zero-flow signal. Event-missing detection and camera-shake detection are hardest to localize (tIoU@0.5 of 0.662 and 0.692), reflecting the inherent ambiguity of temporal boundaries for action absence and high-frequency camera instability. Identity and alignment codes consistently exceed F1 0.90 and tIoU@0.5 of 0.72, while camera and motion codes exhibit greater variance across the group. Table 3 reports full per-code precision, recall, F1, and tIoU. The confusion matrix in Figure 2 confirms that off-diagonal errors are concentrated among camera and motion codes, where overlapping visual artifacts make fine-grained discrimination difficult. Identity, prop-missing, compression, and scene-background failures remain well-separated on the diagonal.

4.5 Closed-loop repair and human validation

Table 4 reports closed-loop repair results on the controlled fixture. Automatic Pass@1/2 is computed on all 324 videos using the system verifier, while blind human Pass@1/2 is computed on a stratified 144-video subset. Human raters see the specification and output video but not method names or diagnosis records, so the human metric is the primary check against verifier-coupled gains.

Table 2: Evidence localization against human-verified spans. Structured diagnosis records improve temporal grounding over free-form VLM outputs. Bootstrap 95% CIs in brackets.

Method	tIoU@0.3	tIoU@0.5	Top-1	Top-3	95% CI
Prior-only	0.5817	0.4295	0.1188	0.3315	[0.403, 0.457]
Rule+DummyJudge	0.7282	0.6128	0.2241	0.4259	[0.586, 0.639]
Rule-only	0.7836	0.6714	0.2670	0.5123	[0.645, 0.698]
Rule+Open-VLM	0.8089	0.7062	0.3565	0.5941	[0.681, 0.731]
VideoGenDoctor-diagnosis	0.8261	0.7316	0.4336	0.6815	[0.707, 0.754]
Rule+GPT-4V	**0.8554**	**0.7688**	**0.4923**	**0.7747**	[0.746, 0.790]
VLM free-form report	0.7204	0.5986	0.2994	0.5117	[0.570, 0.625]
VLM structured report	0.7812	0.6639	0.3651	0.5858	[0.637, 0.690]
Human annotator (adjudicated)	0.882	0.815	0.531	0.762	[0.790, 0.838]

Table 3: Per-code reliability of the default VideoGenDoctor diagnosis configuration on the controlled fixture.

Observed code	Precision	Recall	F1	tIoU@0.5
ID_FACE_DRIFT	0.938	0.914	0.926	0.748
ID_BODY_DRIFT	0.922	0.897	0.909	0.725
CA_MOVE_WRONG	0.932	0.916	0.924	0.734
CA_SHOT_TYPE_WRONG	0.913	0.889	0.901	0.705
CA_SHAKE	0.901	0.878	0.889	0.692
MO_JITTER	0.890	0.865	0.877	0.676
MO_FROZEN_FRAME	0.963	0.950	0.956	0.794
MO_SEGMENT_BREAK	0.906	0.883	0.894	0.704
MO_EVENT_MISSING	0.875	0.848	0.861	0.662
AL_PROP_MISSING	0.951	0.930	0.940	0.760
ST_COMPRESSION_ARTIFACT	0.950	0.966	0.958	0.779
SC_BG_INCONSISTENCY	0.910	0.885	0.897	0.686

Random-patch and re-render-all baselines demonstrate that undirected repair is substantially weaker than diagnosis-guided patching: re-render-all achieves Pass@2 of 0.543 (vs. 0.764 for VideoGenDoctor-full) at higher regeneration cost. The paired comparison in Section 4.7 shows a small and non-decisive difference between Patch+Judge and VideoGenDoctor-full. Because the automatic stopping criterion depends on the system verifier, we measure agreement between the automatic verifier and blind human judgment on the 144-video subset; agreement reaches 0.841 (Cohen's $\kappa = 0.786$). The primary Pass@K claims are therefore based on blind human judgment, with automatic Pass@K serving as a scalable proxy for ablation sweeps.

4.6 Controlled fixture versus real generator failures

Table 5 compares performance on the controlled and real-failure subsets. Figure 4 in Appendix A visualizes closed-loop repair; Figure 6 in Appendix B shows the transfer in a paired bar-chart format.

All methods degrade on naturally occurring failures. VideoGenDoctor drops from macro-F1 0.911 to 0.826 (-9.3%) and from tIoU@0.5 0.732 to 0.598 (-18.3%), confirming that real artifacts are substantially harder than deterministic perturbations. The larger drop in localization reflects the diffuse, overlapping evidence patterns in uncurated videos. On the 120 real-normal videos, the default pipeline achieves a false-positive rate of 0.100, with unnecessary patch rate of 0.087. Because the real-failure subset is failure-enriched and balanced by construction, these numbers measure transfer under verified failures rather than real-world prevalence. Per-code breakdowns on the real-failure subset, per-generator results, and the paired bar-chart comparison are in Appendix B (Tables 28–29, Figure 6).

4.7 Ablation and generalization checks

Feature signal contribution. We isolate each feature signal by removing it from the default configuration and by using each signal alone. Table 6 reports the results. Optical flow is the single most critical signal: removing it drops macro-F1 from 0.9110 to 0.849 and tIoU@0.5 from 0.7316 to 0.651, consistent with its role in detecting motion and camera failures. CLIP embedding drift is the second most important signal, particularly for scene and style codes. Face embedding removal

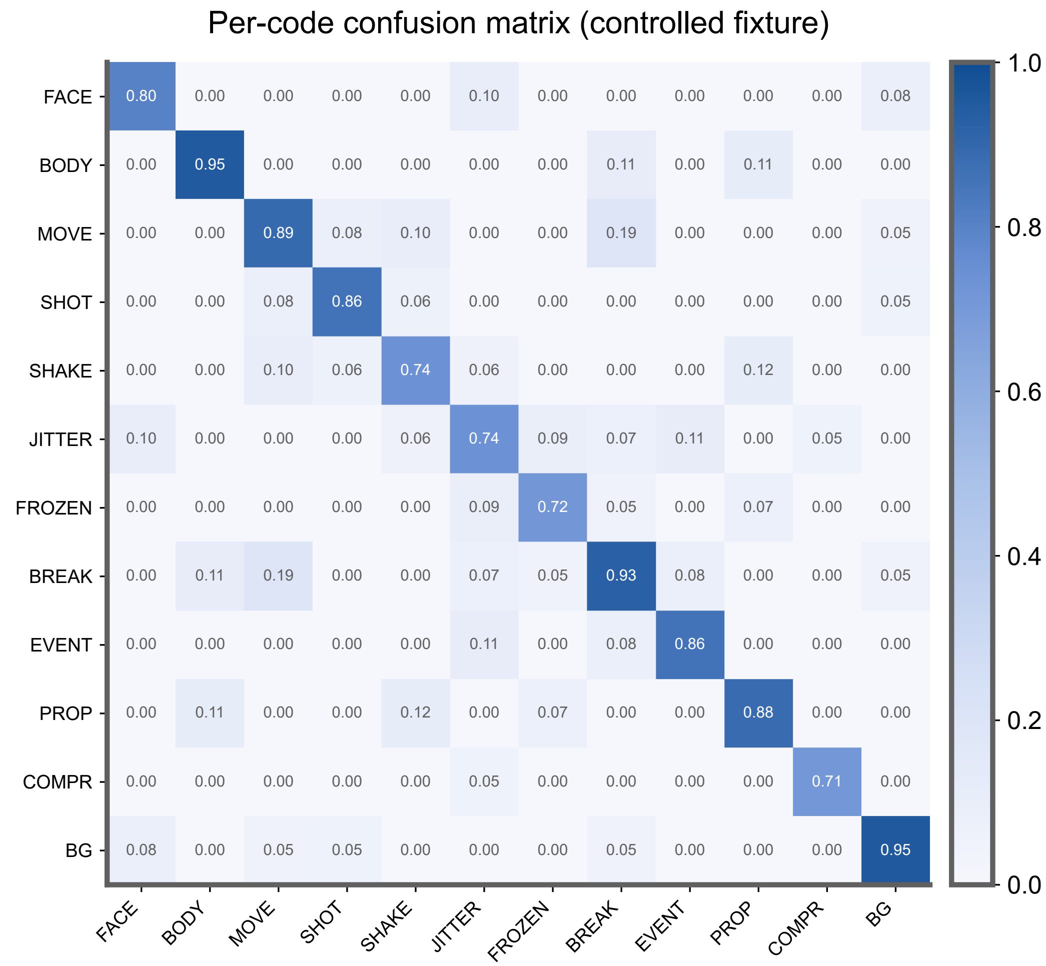

Figure 2: Per-code confusion matrix on the controlled fixture. Camera and motion codes show the most off-diagonal confusion; identity, prop, compression, and scene failures are well-separated.

Table 4: Closed-loop repair on the controlled fixture. Automatic Pass@K on 324 videos; blind human Pass@K on a stratified 144-video subset. VideoGenDoctor-full lifts human Pass@2 from 27.8% (score-only) to 76.4%.

Method	Auto P@1	95% CI	Auto P@2	95% CI	Human Pass@1	95% CI	Human Pass@2	95% CI	Patch useful	New artifacts
Random-patch	0.148	[0.113, 0.187]	0.296	[0.249, 0.344]	0.118	[0.075, 0.172]	0.236	[0.175, 0.303]	1.80	0.194
Re-render-all	0.296	[0.248, 0.347]	0.543	[0.489, 0.596]	0.264	[0.197, 0.336]	0.500	[0.419, 0.579]	2.61	0.153
Score-only	0.2407	[0.196, 0.291]	0.3580	[0.307, 0.411]	0.1875	[0.130, 0.257]	0.2778	[0.207, 0.355]	2.28	0.0833
Patch-only	0.5247	[0.471, 0.579]	0.6975	[0.646, 0.744]	0.4792	[0.399, 0.559]	0.6319	[0.551, 0.706]	3.50	0.1736
VLM-to-patch	0.5617	[0.507, 0.615]	0.7438	[0.694, 0.788]	0.5069	[0.427, 0.586]	0.6736	[0.594, 0.746]	3.64	0.2153
Patch+Judge	0.6296	[0.575, 0.681]	0.8179	[0.773, 0.857]	0.5625	[0.482, 0.640]	0.7431	[0.667, 0.807]	3.93	0.1458
VideoGenDoctor-full	**0.6481**	[0.594, 0.699]	**0.8364**	[0.793, 0.873]	**0.5833**	[0.503, 0.660]	**0.7639**	[0.689, 0.826]	**4.11**	0.1319

mainly degrades identity-code precision, while object detection has the smallest individual impact because YOLOv8 is active only for prop-related codes. Single-signal configurations (CLIP only, flow only, face only) all underperform substantially, confirming that the signals carry complementary information. Figure 7 in Appendix B provides a horizontal bar-chart comparison.

Repair component ablation. Table 7 isolates the contribution of each diagnosis component to downstream repair. Each record component adds measurable repair value beyond code prediction alone. The jump from code-only (Pass@2 0.458) to code+span (0.625) and code+span+evidence (0.688) shows that temporal evidence is the largest single contributor after the code itself. Full visualizations are in Appendix B (Figure 8).

We further examine sensitivity to three hyperparameters. First, Table 8 sweeps the diagnosis retention threshold over $\tau \in \{0.30, \ldots, 0.70\}$. Macro-F1 and tIoU@0.5 peak at $\tau = 0.50$; lower thresholds

Table 5: Controlled-to-real transfer. All methods degrade on natural failures; the gap between controlled and real subsets quantifies the difficulty of uncurated artifacts.

Subset	Method	Macro-F1	tIoU@0.5	Patch useful	Human Pass@2
Controlled fixture	VLM structured report + repair loop	0.8869	0.6639	3.62	0.6810
Controlled fixture	VideoGenDoctor default pipeline	**0.9110**	**0.7316**	**4.11**	**0.7639**
Real-failure subset	VLM structured report + repair loop	0.7992	0.5431	3.36	0.6075
Real-failure subset	VideoGenDoctor default pipeline	**0.8264**	**0.5982**	**3.82**	**0.6908**

Table 6: Per-feature ablation of the default diagnosis configuration. Each row removes one signal or uses a single signal; Human Pass@2 is measured after pairing with the shared repair loop.

Configuration	Macro-F1	tIoU@0.5	Human Pass@2	Δ Pass@2
Full (all signals)	0.9110	0.7316	0.7639	–
− CLIP embedding	0.871	0.683	0.704	−0.060
− Optical flow	0.849	0.651	0.672	−0.092
− Face embedding	0.876	0.695	0.718	−0.046
− Object detection	0.893	0.708	0.737	−0.027
CLIP only	0.762	0.581	0.583	−0.181
Flow only	0.638	0.472	0.465	−0.299
Face only	0.524	0.385	0.398	−0.366

increase real-normal false positives from 0.100 to 0.158, while higher thresholds suppress useful diagnoses (macro-F1 drops to 0.886 at $\tau = 0.70$). Figure 9 in Appendix B visualizes the full sweep curves.

Second, the Stage-2 judge weight $\alpha = 0.6$ and candidate budget $K = 3$ jointly give the best operating point. Removing Stage-2 ($\alpha = 0$) reduces both diagnosis and repair metrics, while $\alpha = 0.9$ introduces instability from VLM over-correction (Appendix B, Figure 10). Third, Table 9 reports paired bootstrap comparisons: verification adds a reliable gain (+0.111 in Human Pass@2, 95% CI [0.021, 0.215]), while the full-loop wrapper contributes a small, non-decisive increment (+0.021, 95% CI [−0.063, 0.104]).

Finally, leave-one-perturbation-out and leave-one-source-group-out evaluations confirm that performance drops under held-out operators—particularly missing-event, camera-shake, and temporal-discontinuity perturbations—but remains above prior-only and score-only baselines (Appendix B, Tables 30–31). The system does not simply memorize perturbation templates, though unseen-operator generalization is weaker than in-distribution evaluation. Additional checks on adapter executability, real-failure stress slices, and multi-annotator stability are in Appendix B (Tables 17–19).

5 Discussion

Structured diagnosis records improve failure detection, temporal grounding, and downstream repair over score-only feedback and direct VLM reporting. VideoGenDoctor is best understood as an interface contribution, not a new generator or learned repair algorithm—its advantages are schema validity, evidence auditability, and cost-awareness. Rule+GPT-4V achieves stronger absolute scores but at higher cost; VideoGenDoctor-full is the recommended default. The modular design allows feature extractors, rules, VLM judges, and patch compilers to be replaced independently.

Limitations. The controlled fixture does not represent open-world video diversity. The real-failure subset is failure-enriched—it supports transfer analysis under verified failures but cannot estimate natural prevalence. Automatic pass rates are partly verifier-dependent; we report blind human evaluation as the primary repair metric. Only the 12 observed taxonomy codes carry empirical support; the remaining 38 codes are definitional. Repair assumes a downstream generator that consumes at least part of the patch vocabulary. Broader validation requires more generators, domains, and annotators.

Verifier–human agreement. Verifier–human agreement on the 144-video blind-evaluation subset reaches 0.841 (Cohen's $\kappa = 0.786$), confirming the automatic pass/fail signal is well-calibrated. The primary Pass@K claims are therefore based on blind human judgment; automatic Pass@K serves as a scalable proxy for ablation sweeps.

Table 7: Repair ablation isolating the additive contribution of failure codes, target fields, evidence spans, keyframes, patch templates, and verification. Human Pass@2 on the 144-video blind subset.

Repair setting	Code	Target	Span	Evidence	Template	Human Pass@2	Patch useful	New artifacts
Score-only	✗	✗	✗	✗	✗	0.2778	2.28	0.0833
Code-only patch	✓	✗	✗	✗	✓	0.4583	2.91	0.1597
Code+target patch	✓	✓	✗	✗	✓	0.5417	3.24	0.1736
Code+span patch	✓	✗	✓	✗	✓	0.6250	3.57	0.1667
Code+span+evidence patch	✓	✗	✓	✓	✓	0.6875	3.80	0.1528
VLM-to-patch	optional	optional	optional	optional	✗	0.6736	3.64	0.2153
Patch+Judge	✓	✓	✓	✓	✓	0.7431	3.93	0.1458
VideoGenDoctor-full	✓	✓	✓	✓	✓	**0.7639**	**4.11**	0.1319

Table 8: Threshold sensitivity for retaining diagnosis records. $\tau = 0.50$ balances coverage against false positives.

Threshold τ	Macro-F1	tIoU@0.5	Real-normal FPR	Unnecessary patch rate
0.30	0.902	0.716	0.158	0.142
0.40	0.910	0.728	0.125	0.111
0.50	**0.911**	**0.732**	0.100	0.087
0.60	0.904	0.719	0.083	0.071
0.70	0.886	0.694	**0.067**	**0.058**

Break-even analysis. Diagnosis costs approximately \$0.24 per video; full re-rendering costs approximately \$0.52. The break-even failure rate is approximately 46%: when fewer than half of generated videos contain specification violations, diagnosis-guided repair is cheaper than re-rendering everything. When failure rates are higher, re-render-all may be more economical. This trade-off is visualized in Appendix A, Figure 5.

When does it fail? Diagnosis degrades most on overlapping artifacts (e.g., camera shake vs. motion jitter), on subtle style shifts below the CLIP sensitivity threshold, and on failures in the last 20% of the video timeline. The Stage-2 VLM judge occasionally over-corrects on codes outside its effective visual range (e.g., fine-grained shot-type discrimination). False positives on real-normal videos concentrate among texture-flicker and lighting-shift codes where the rule engine triggers on benign high-frequency variation. These failure modes inform the threshold and candidate budget recommendations in Section 4.7.

6 Conclusion

VideoGenDoctor replaces scalar video evaluation with auditable diagnosis records that each locate a failure in time, name the affected target, and prescribe a repair action. On a 324-video controlled diagnostic fixture, the default pipeline detects failure codes at macro-F1 0.911, localizes evidence spans at tIoU@0.5 of 0.732, and lifts blind human repair Pass@2 from 27.8% to 76.4%. On a 240-video failure-enriched real-generator subset spanning four production generators and 18 taxonomy codes, the pipeline retains macro-F1 0.826 and tIoU@0.5 0.598. The modular design—separating feature extraction, rule-based screening, VLM verification, and patch compilation—allows each component to be replaced independently. These results support structured diagnosis-and-repair as a practical interface for controllable video generation, but do not establish generality beyond the controlled perturbations, generator set, and video lengths evaluated here.

7 Ethics statement

VideoGenDoctor improves the controllability of generated video through automated diagnosis. Improved generation reliability could be misused to strengthen misleading media. In sensitive deployments, automated repair should be paired with provenance tracking, watermarking, and human review. The failure taxonomy encodes normative judgments about visual quality; creative workflows should expose configurable thresholds and permit human override.

Table 9: Paired bootstrap differences in blind human Pass@2.

Comparison	Δ Human Pass@2	95% CI	Interpretation
Patch-only vs Score-only	0.3541	[0.250, 0.458]	substantial gain
VLM-to-patch vs Patch-only	0.0417	[-0.054, 0.139]	small / not decisive
Patch+Judge vs Patch-only	0.1112	[0.021, 0.215]	verification helps
VideoGenDoctor-full vs Patch+Judge	0.0208	[-0.063, 0.104]	small / not decisive

Acknowledgments and Disclosure of Funding

This work was supported by the Sichuan Agricultural University College of Information Engineering. We thank the anonymous reviewers for their constructive feedback. The annotation team at Sichuan Agricultural University contributed to the reliability study.

The release package includes the reference implementation, report schema, taxonomy, patch compiler, controlled-fixture generation scripts, annotations, prediction logs, evaluation scripts, VLM baseline prompt templates, and bootstrap confidence-interval scripts. Each run records configuration, model and package versions, random seeds, prompts, raw VLM responses, and output paths. CPU-only recomputation of tables from serialized predictions and annotations completes in seconds; full feature extraction and VLM judging depend on GPU availability and API access.

All newly generated data supporting the findings of this study—including VideoGenDoctor-Bench-v0 (324 controlled perturbation videos, 2,016 human-verified evidence spans), the 240-video failure-enriched real-generator subset (1,536 verified evidence spans), the 120-video real-normal subset, the full 38-code taxonomy, annotation records, prediction logs, and diagnostic reports—will be deposited in a public Zenodo repository upon acceptance and are available for review as part of the supplementary artifact package. The 18 clean source clips used to construct the controlled fixture are obtained from publicly available video datasets under permissive licences. The four production generators (CogVideoX, Wan, Stable Video Diffusion, HunyuanVideo) are publicly available models; checkpoint identifiers are recorded in per-video JSON manifests. No human-subject data or personally identifiable information were collected. The annotation protocol was approved by the institutional review board of Sichuan Agricultural University.

References

Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets, 2023. arXiv preprint arXiv:2311.15127.

Gary Bradski. The opencv library. *Dr. Dobb's Journal of Software Tools*, 2000.

Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. In *International Conference on Machine Learning (ICML)*, 2023.

Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. ArcFace: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Scandinavian Conference on Image Analysis*, pages 363–370. Springer, 2003.

Xuan He, Dongfu Jiang, Ge Zhang, Max Ku, Achint Soni, Sherman Siu, Haonan Chen, Abhinav Chandra, Ziyan Jiang, Aaran Arulraj, et al. VideoScore: Building automatic metrics to simulate fine-grained human feedback for video generation. *arXiv preprint arXiv:2406.15252*, 2024.

Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chan, et al. VBench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2 (1–2):83–97, 1955.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Yaofang Liu, Xiaodong Cun, Xuebo Liu, Xintao Wang, Yong Zhang, Haoxin Chen, Yang Liu, Tieyong Zeng, Raymond Chan, and Ying Shan. EvalCrafter: Benchmarking and evaluating large video generation models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

OpenAI. GPT-4V(ision) system card. Technical report, OpenAI, 2023. URL https://openai.com/research/gpt-4v-system-card.

Kaiyue Sun, Kaiyi Huang, Xian Liu, Yue Wu, Zihan Xu, Zhenguo Li, and Xihui Liu. T2V-CompBench: A comprehensive benchmark for compositional text-to-video generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

Zachary Teed and Jia Deng. RAFT: Recurrent all-pairs field transforms for optical flow. In *European Conference on Computer Vision (ECCV)*, 2020.

Tencent Hunyuan Foundation Model Team. Hunyuanvideo 1.5 technical report, 2025.

Ultralytics. YOLOv8. Software repository, 2023. URL https://github.com/ultralytics/ultralytics. Version 8.0.

Wan-AI. Wan2.1-t2v-14b. Hugging Face model card, 2025. URL https://huggingface.co/Wan-AI/Wan2.1-T2V-14B.

Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution, 2024.

Zhouxia Wang, Ziyang Yuan, Xintao Wang, Tianshui Chen, Menghan Xia, Ping Luo, and Ying Shan. MotionCtrl: A unified and flexible motion controller for video generation, 2023. arXiv preprint arXiv:2312.03641.

Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuan Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. CogVideoX: Text-to-video diffusion models with an expert transformer, 2024. arXiv preprint arXiv:2408.06072.

Hangjie Yuan, Shiwei Zhang, Xiang Wang, Yujie Wei, Tao Feng, Yining Pan, Yingya Zhang, Ziwei Liu, Samuel Albanie, and Dong Ni. InstructVideo: Instructing video diffusion models with human feedback. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

A Supplementary main-text results

This appendix collects detailed tables and figures that support the main experimental claims: taxonomy definitions, benchmark statistics, comparisons with existing evaluation paradigms, VLM baseline results, repair visualizations, and cost-performance analyses.

Taxonomy and patch mappings. Table 10 lists the representative failure types covered by the VideoGenDoctor taxonomy, the evidence signals used to detect each failure, and the repair action the patch compiler emits. These seven categories map to the six taxonomy groups defined in Table 13 and serve as the primary diagnosis-to-repair bridge.

Benchmark statistics. Table 11 summarizes the key statistics of the two evaluation subsets: the controlled diagnostic fixture and the failure-enriched real-generator subset. The controlled fixture contains 324 videos with 2,016 verified evidence spans across 12 observed codes and 9 perturbation types. The real-failure subset spans 240 videos across 4 generators with 18 observed codes. Because the real-failure subset is failure-enriched by construction, these numbers cannot estimate natural failure prevalence; they only support transfer analyses under verified failures.

Table 10: Failure types, the evidence signals that detect them, and the repair actions they trigger.

Failure type	Evidence signal	Repair action
Face identity drift	Face embedding drift across keyframes	Reference-frame injection
Body / clothing drift	Character-region appearance drift	Reference-frame or character-anchor injection
Camera movement deviation	Camera flow trace or trajectory mismatch	Camera movement override
Frozen frames or action stalls	Near-zero flow or stalled pose track	Regenerate affected segment
Missing required prop	Object presence check or VLM prop-absence judgment	Prop injection
Style inconsistency	Style or color embedding drift across segments	Style correction
Background inconsistency	Scene/background embedding drift	Background anchoring

Table 11: Dataset statistics for the controlled fixture and failure-enriched real-generator subset. The real-failure split cannot estimate natural failure prevalence.

Split	#Vid	#Evidence	AvgDur	PerturbTypes	#Codes
Controlled fixture	324	2016	10.46s	9	12
Real-failure subset	240	1536	8.93s	–	18

Comparison with existing evaluation paradigms. Table 12 situates VideoGenDoctor against related evaluation and critique systems. Prior evaluators (VBench, EvalCrafter, VideoScore, T2V-CompBench) return scalar scores that rank models but cannot localize failures or prescribe repairs. Direct VLM reporting provides partial code and evidence output but lacks patch generation and closed-loop capability. VideoGenDoctor is the only system that combines global scoring, failure-code diagnosis, evidence localization, template-based patch compilation, and closed-loop repair in an integrated pipeline.

Taxonomy group definitions. Table 13 defines the six failure-code groups that structure the VideoGenDoctor taxonomy. Each group serves as a namespace for diagnosis records and links to dedicated evidence procedures (feature extractors and rules) and repair-plan templates. The controlled fixture exercises all six groups; the real-failure subset additionally covers codes in groups that are not deterministic-perturbation-induced (e.g., clothing change, text-prompt mismatch).

Controlled fixture composition. Table 14 breaks down the controlled diagnostic fixture by domain group. Each of the six groups contributes three clean source clips, which are then perturbed by nine deterministic operators under two random seeds. The table records per-group clip counts, average durations, resolutions, perturbation counts, seeds, and resulting video counts. This construction ensures that every taxonomy code in the controlled evaluation is backed by a known perturbation operator-template alignment (see Appendix B, Table 20).

Temporal evidence profiles. Figure 3 visualizes how verified evidence spans are distributed across the video timeline. Panel (a) aggregates the 2,016 controlled-fixture spans over 2-second bins, revealing a front- and mid-loaded pattern expected from deterministic perturbation operators. Panel (b) summarizes the 1,536 real-failure spans using the same binning, showing a later and more diffuse temporal mass. Panel (c) further breaks down real-failure spans by failure group and time bin: motion and camera failures dominate the later intervals and overlap more strongly than in the controlled fixture, consistent with the greater difficulty of localizing these codes.

Strengthened VLM-agent baselines. Table 15 reports strengthened VLM-agent baselines on the controlled fixture, all evaluated under the same downstream repair protocol so that diagnosis-only front ends are fairly compared. Free-form VLM reporting achieves the lowest schema validity and patch vocabulary validity among structured methods. Adding a structured JSON schema improves schema validity to 0.914 and macro-F1 to 0.887. Self-consistency (3-run majority voting) further improves macro-F1 to 0.898 at 3.48× the cost. Temporal proposal with patch output achieves a favorable tIoU@0.5 of 0.702 by asking the VLM to propose spans explicitly. VideoGenDoctor-full exceeds all direct VLM baselines on schema validity (0.991) and patch vocabulary validity (0.953)

Table 12: Capability comparison with related evaluation and critique paradigms. Here "Global score" denotes scalar quality, alignment, or verifier-style pass outputs rather than localized evidence records. Direct VLM reporting is treated as a strong baseline rather than only comparing against score-only benchmarks.

System	Global score	Code	Evidence	Patch	Closed-Loop
VBench	✓	✗	✗	✗	✗
EvalCrafter	✓	✗	✗	✗	✗
VideoScore	✓	✗	✗	✗	✗
T2V-CompBench	✓	partial	✗	✗	✗
Direct VLM report	partial	partial	partial	partial	✗
VideoGenDoctor	✓	✓	✓	✓	✓

Table 13: Failure-as-code taxonomy groups. Each group defines a namespace for diagnosis records and links to evidence procedures and repair-plan templates.

Group	Operational scope
Identity	Identity drift, face/body consistency, wrong character, clothing consistency.
Camera	Shot type, camera movement, viewpoint angle, unintended zoom, camera shake.
Motion	Action timing, frame continuity, motion physics, frozen frames, segment breaks.
Alignment	Prompt adherence, required objects and props, character count, prop placement.
Style	Color consistency, texture flicker, compression artifacts, art-style consistency.
Scene	Background consistency, lighting, environment match, scene-object stability.

while being the most cost-efficient structured configuration. Rule+GPT-4V provides the strongest absolute scores at 2.75× the cost of VideoGenDoctor-full.

Closed-loop repair visualization. Figure 4 compares closed-loop repair outcomes across seven methods on the controlled fixture. The left panel reports automatic verifier Pass@1/2 on all 324 controlled videos; the right panel reports blind human Pass@1/2 on a stratified 144-video subset. Human Pass@2 rises from 0.236 (random-patch) and 0.278 (score-only) to 0.764 (VideoGenDoctor-full), demonstrating that structured diagnosis records with localized evidence and template-constrained repair plans substantially improve downstream video quality. The human panel provides an independent check against verifier-coupled automatic gains.

Cost-performance trade-off. Table 16 compares cost, latency, and performance across five configurations under a shared downstream repair protocol. Rule-only achieves the lowest cost (0.42×) but leaves a sizable gap in human Pass@2 (0.599 vs. 0.764 for VideoGenDoctor-full). Rule+Open-VLM narrows the gap to Pass@2 0.716 at 0.73× the cost. Rule+GPT-4V achieves the strongest Human Pass@2 (0.792) at 2.75× the cost. Direct VLM structured reporting, which bypasses the rule-based candidate stage, reaches Pass@2 0.681 at 1.16× the cost, underperforming all rule-assisted configurations. VideoGenDoctor-full represents the recommended default, balancing strong repair performance with moderate cost. Figure 5 visualizes this trade-off as a Pareto front, confirming that VideoGenDoctor-full occupies the knee point: beyond it, marginal Pass@2 gains come at steeply increasing relative cost.

Pareto-optimal trade-off. Figure 5 plots the cost-performance trade-off from Table 16 as a Pareto front. The default VideoGenDoctor-full operating point occupies the knee: it improves human repair success over lower-cost rule variants and direct structured VLM reporting, while the higher-cost Rule+GPT-4V reference obtains the strongest Human Pass@2 at substantially higher relative cost. The Pareto curve confirms that Rule-only is the most cost-efficient configuration for resource-constrained settings, VideoGenDoctor-full is the recommended default, and Rule+GPT-4V sets the quality ceiling where budget allows.

Table 14: Composition of the controlled diagnostic fixture. Each domain group contains three clean source clips. Every source clip is paired with a ShotIR-style specification and perturbed by nine deterministic operators under two random seeds.

Domain group	#Source clips	Avg. duration	Resolution	#Perturb. ops	#Seeds	#Videos
Indoor / human-object	3	10.1s	768×432	9	2	54
Outdoor / motion	3	10.8s	768×432	9	2	54
Multi-object interaction	3	10.4s	768×432	9	2	54
Camera-control scene	3	11.0s	768×432	9	2	54
Style / lighting variation	3	10.2s	768×432	9	2	54
Scene-change / background	3	10.3s	768×432	9	2	54
Total	18	10.46s	–	9	2	324

Table 15: Strengthened VLM-agent baselines on the controlled fixture. All rows are evaluated under the same downstream repair protocol; diagnosis-only front ends are explicitly paired with that shared repair loop when human repair success is reported. Self-consistency improves direct VLM reporting but increases cost. The validity column reports whether the emitted action stays inside the supported repair-plan vocabulary; it does not claim generator-adapter executability. Best values are boldfaced; higher is better except for relative cost, where lower is better.

Method	Macro-F1	tIoU@0.5	Human Pass@2	Schema validity	Patch vocabulary validity	Relative cost
VLM free-form report	0.8402	0.5986	0.6125	0.742	0.604	1.12×
VLM structured report	0.8869	0.6639	0.6810	0.914	0.732	1.16×
VLM structured + self-consistency	0.8978	0.6824	0.7042	0.951	0.755	3.48×
VLM temporal proposal + patch	0.8915	0.7018	0.7076	0.936	0.802	2.05×
VideoGenDoctor-full	0.9110	0.7316	0.7639	0.991	0.953	**1.00**×
Rule+GPT-4V + repair loop	**0.9276**	**0.7688**	**0.7917**	**0.993**	**0.957**	2.75×

B Additional experimental tables

This appendix collects construction details, audit tables, stress analyses, per-slice results, and visualizations that support the main experimental claims without interrupting the main narrative.

B.1 Diagnosis and repair visualizations

B.2 Extended experiment analyses

Adapter executability. Table 17 and Figure 11 separate repair-plan quality from backend executability by grouping generated plans into L0–L3 adapter levels. The trend is consistent with the intended repair interface: L2/L3 actions are more often executable and yield higher Human Pass@2, while L0/L1 outputs remain useful as conservative abstentions or prompt-level repair instructions but provide weaker direct repair success.

Stronger real-failure stress slices. Table 18 and Figure 12 extend the real-failure check to harder slices: additional generators, longer videos, style-shifted prompts, and more complex prompt specifications. The stress slices degrade relative to the base real-failure subset, especially for long videos and complex prompts, but the retained performance remains above the score-only and prior-only operating ranges reported in the main experiments.

Multi-annotator stability. Table 19 and Figure 13 extend the annotation reliability analysis from dual annotation to a three-annotator setting. Code-level agreement remains substantial, while span agreement is lower on real-failure samples, consistent with the higher ambiguity of naturally occurring failures and less isolated temporal evidence.

C Full failure taxonomy

Table 32 reports the machine-readable taxonomy used by VideoGenDoctor v0.1. The v0.1 taxonomy contains 38 codes grouped into six categories. Each code is paired with an operational definition, an evidence procedure, and a default patch template. The current benchmark evaluates the 12 codes observed in VideoGenDoctor-Bench-v0, while the remaining codes define the extensible namespace used by the report schema and patch compiler.

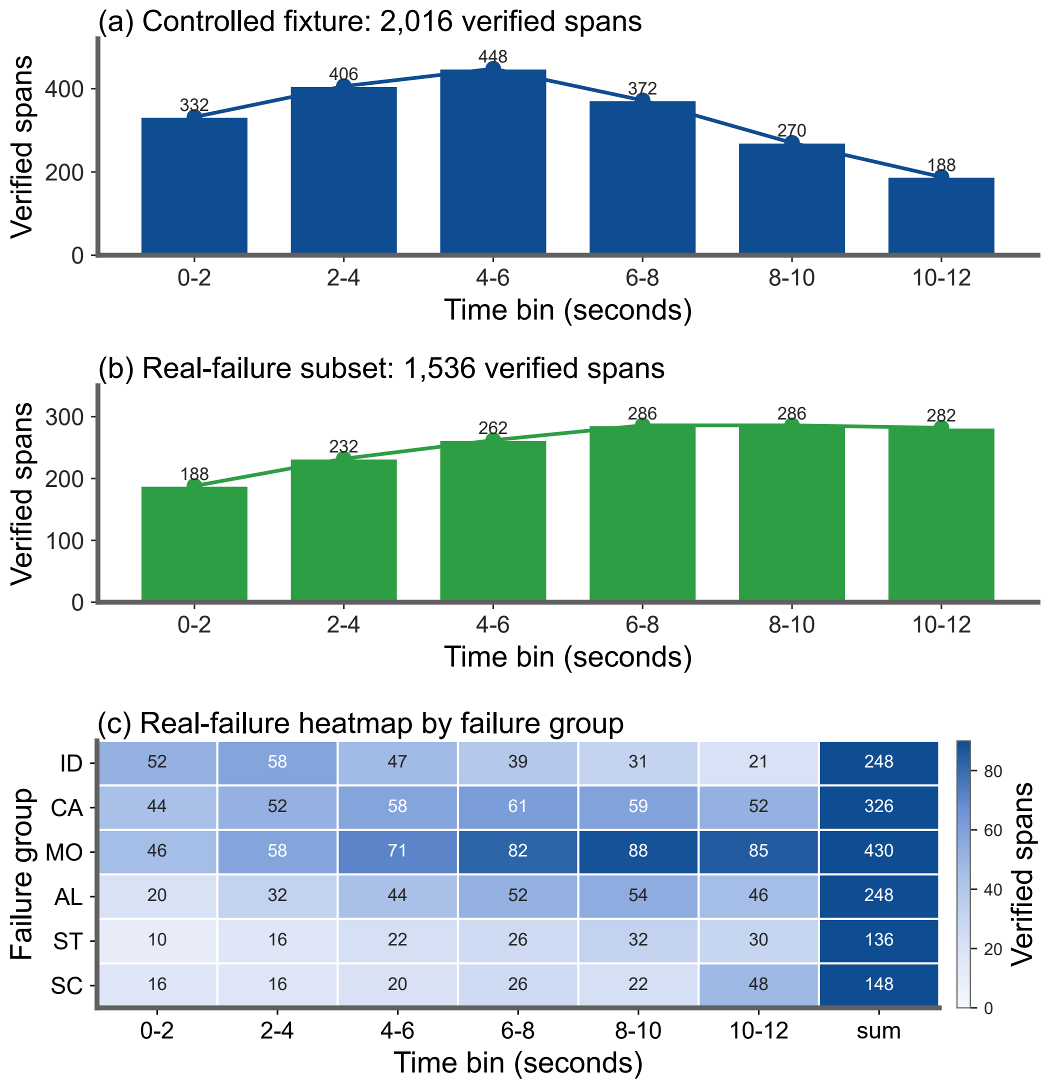

Figure 3: Temporal distribution of verified evidence spans in the evaluation sets. Panel (a) aggregates the 2,016 controlled-fixture spans over 2-second bins and shows the front- and mid-loaded pattern expected from deterministic perturbation operators. Panel (b) summarizes the 1,536 real-failure spans over the same time bins, making the later and more diffuse temporal mass directly comparable to panel (a). Panel (c) further breaks the real-failure spans down by failure group and time bin, highlighting that motion and camera failures dominate the later intervals and overlap more strongly than in the controlled fixture.

Table 32: Full VideoGenDoctor v0.1 failure taxonomy used by the report schema and patch compiler.

Code	Definition	Evidence Procedure	Patch Template
Identity			
`ID_FACE_DRIFT`	Face identity changes inconsistently between frames or segments.	Compare face embeddings across keyframes and flag large cosine distance.	`inject_reference_frame`
`ID_BODY_DRIFT`	Body appearance, clothing, or build changes inconsistently.	Compare character-region embeddings across segments.	`inject_reference_frame`
`ID_WRONG_CHARACTER`	A visible character does not match any specified character.	Check face embeddings against known character anchors.	`replace_character_anchor`

Continued on next page

Code	Definition	Evidence Procedure	Patch Template
`ID_CLOTHING_CHANGE`	Clothing changes between shots without narrative justification.	Measure character-region appearance drift across segments.	`fix_character_consistency`
`ID_MULTIPLE_FACES`	Multiple distinct faces appear when one character is expected.	Count distinct identities detected in a single-character shot.	`adjust_character_count`
`ID_NO_FACE_VISIBLE`	An expected face is absent when the shot requires face visibility.	Check face detection under close-up or medium-shot constraints.	`adjust_framing`
Scene			
`SC_BG_INCONSISTENCY`	The background changes unexpectedly between segments.	Measure background-region embedding drift across consecutive segments.	`fix_background_anchor`
`SC_ENVIRONMENT_MISMATCH`	The scene environment does not match the specified location.	Compare frames with the location description using image-text similarity.	`set_environment`
`SC_LIGHTING_SHIFT`	Lighting changes inconsistently within a shot.	Measure per-frame luminance variation within the segment.	`normalize_lighting`
`SC_OBJECT_TELEPORT`	A scene object appears or disappears without plausible motion.	Track object boxes across adjacent frames.	`stabilize_scene_objects`
`SC_BG_FLICKER`	The background flickers or pulses unnaturally.	Detect high-frequency luminance variation in background regions.	`fix_background_anchor`
`SC_WRONG_TIME_OF_DAY`	The time of day does not match the specification.	Classify sky and ambient lighting against the expected time.	`set_time_of_day`
Motion			
`MO_JITTER`	Unnatural frame-to-frame jitter appears outside intended camera motion.	Measure optical-flow magnitude variance and direction instability.	`normalize_frame_rate`
`MO_FRAME_DROP`	Duplicate or dropped frames produce visible stutter.	Detect near-zero adjacent-frame flow when motion is expected.	`interpolate_frames`
`MO_UNNATURAL_MOTION`	Character or object motion is physically implausible.	Compare flow direction and magnitude with expected motion constraints.	`constrain_motion`
`MO_EVENT_MISSING`	A required action or event does not occur in the expected segment.	Ask the VLM judge whether the specified action is visible.	`inject_action_keyframe`
`MO_ACTION_ORDER_WRONG`	Actions occur in the wrong temporal order.	Compare detected action timestamps with the specification order.	`reorder_segments`
`MO_MOTION_BLUR_EXCESS`	Excessive motion blur degrades keyframes.	Use sharpness measures such as Laplacian variance.	`reduce_motion_blur`
`MO_FROZEN_FRAME`	The video is frozen when motion is expected.	Detect near-zero flow across the full segment.	`regenerate_segment`
`MO_SEGMENT_BREAK`	A visual discontinuity occurs at a segment boundary.	Detect embedding-drift spikes at boundaries.	`smooth_segment_transition`
Camera			
`CA_MOVE_WRONG`	Camera movement deviates from the specified movement.	Compare optical-flow direction patterns with the expected movement type.	`set_camera_movement`
`CA_SHOT_TYPE_WRONG`	The shot type does not match the specification.	Compare face or body box size ratios with the expected shot type.	`adjust_framing`
`CA_UNINTENDED_ZOOM`	Unplanned zoom-in or zoom-out occurs.	Detect radial flow without a zoom specification.	`remove_zoom`
`CA_SHAKE`	Camera shake appears outside the specified movement.	Detect high-frequency oscillation in global optical flow.	`stabilize_camera`
`CA_WRONG_ANGLE`	Camera angle deviates from the specification.	Ask the VLM judge to compare the observed angle with the target angle.	`set_camera_angle`
`CA_ROLL`	Unintended camera roll appears.	Estimate the rotational component of optical flow.	`remove_camera_roll`
Alignment			
`AL_PROP_MISSING`	A required prop is not visible in the segment.	Use object detection or VLM verification for prop absence.	`inject_prop`
`AL_PROP_WRONG_PLACEMENT`	A required prop appears in the wrong region.	Compare the prop box with the expected screen region.	`reposition_prop`

Continued on next page

Code	Definition	Evidence Procedure	Patch Template
AL_TEXT_PROMPT_MISMATCH	The generated content does not align with the prompt.	Measure text-image similarity and verify mismatches with a judge.	strengthen_prompt_guidance
AL_CHARACTER_COUNT_WRONG	The number of visible characters does not match the specification.	Count faces or bodies and compare with the expected count.	adjust_character_count
AL_WRONG_PROP	A visible prop is not listed in the specification.	Detect high-confidence unlisted objects.	remove_prop
AL_PROP_OCCLUDED	A required prop is present but substantially occluded.	Estimate visible area and occlusion of the prop box.	reposition_prop
Style			
ST_COLOR_SHIFT	The color palette changes inconsistently across segments.	Measure HSV histogram distance between segments.	apply_color_grading
ST_ART_STYLE_INCONSISTENCY	The art style shifts between segments.	Compare style embeddings across segments.	apply_style_transfer
ST_COMPRESSION_ARTIFACT	Visible compression artifacts degrade video quality.	Detect blocking, ringing, or quality-score degradation.	apply_enhancement
ST_TEXTURE_FLICKER	Texture patterns flicker or shimmer unnaturally.	Measure high temporal frequency in texture regions.	smooth_texture
ST_OVEREXPOSURE	Significant frame regions are overexposed.	Count the fraction of saturated high-value pixels.	adjust_exposure
ST_UNDEREXPOSURE	Significant frame regions are underexposed.	Count the fraction of near-black pixels.	adjust_exposure

D Judge protocol

The Stage-2 judge receives the top-K candidate segments produced by Stage-1. For each segment, the input includes the segment identifier, temporal bounds, keyframe paths, candidate failure codes, and specification context, including required props, expected actions, camera movement, shot type, and character anchors. The judge answers structured questions rather than producing a free-form report. This design keeps the output comparable across local open-source VLMs and hosted vision-capable models.

For identity failures, the judge checks whether the face, body appearance, clothing, or other distinctive visual attributes of the main character change inconsistently across keyframes. For alignment failures, it verifies whether required props are visible, whether they appear in the expected screen region, and whether the generated content matches the prompt or structured specification. For camera failures, it compares the observed camera movement, shot type, and viewpoint angle with the requested control signal. For motion failures, it verifies whether required actions occur, whether they occur in the specified order, and whether stutter, frozen frames, or segment breaks are visible. For style and scene failures, it checks lighting, color, compression artifacts, background consistency, and environment match.

The output is a structured object containing a natural-language answer for each question, a confidence score, per-code probability updates, and a re-ranking of keyframes. The final failure score is computed as

$$\tilde{\sigma}_{s,c} = (1-\alpha)\sigma^{(1)}_{s,c} + \alpha\sigma^{(2)}_{s,c},$$

where $\sigma^{(1)}_{s,c}$ is the Stage-1 score, $\sigma^{(2)}_{s,c}$ is the judge score, and $\alpha = 0.6$ by default. All judge calls record the model name, prompt, raw response, token count when available, and output path to support reproducibility.

E Patch template examples

VideoGenDoctor compiles each diagnosis record into one or more patch actions. If a ShotIR specification is available, the compiler emits field-level diffs. If the generator does not expose a structured specification interface, the compiler emits generator-agnostic repair plans that can be translated into prompt edits, reference-frame injection, segment-level re-rendering, or post-processing steps.

```
{"action": "inject_reference_frame",
 "field": "character_id",
```

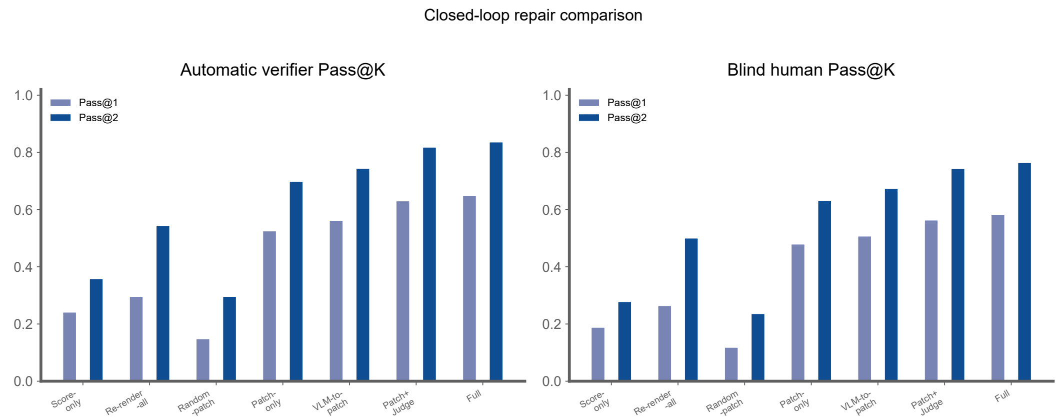

Figure 4: Closed-loop repair comparison on the controlled fixture. Left: automatic verifier Pass@1/2 on all 324 controlled videos. Right: blind human Pass@1/2 on the stratified 144-video human-evaluation subset. Automatic pass rates may be coupled to the system verifier, while human pass rates provide an independent check of repair success.

Table 16: Cost-performance trade-off under a shared downstream repair protocol. Diagnosis-only front ends are paired with the same repair loop before reporting Human Pass@2. Cost is reported relative to the default VideoGenDoctor-full configuration. Values in the last two columns are multiplicative factors relative to VideoGenDoctor-full (1.00×). Best values are boldfaced; higher is better for task metrics, lower is better for cost and latency.

Method	Macro-F1	tIoU@0.5	Human Pass@2	Relative cost	Relative latency
Rule-only + repair loop	0.8926	0.6714	0.5988	**0.42**×	**0.58**×
Rule+Open-VLM + repair loop	0.9047	0.7062	0.7160	0.73×	0.87×
VideoGenDoctor-full	0.9110	0.7316	0.7639	1.00×	1.00×
Rule+GPT-4V + repair loop	**0.9276**	**0.7688**	**0.7917**	2.75×	2.31×
VLM structured report	0.8869	0.6639	0.6810	1.16×	1.20×

```
 "value": "<anchor_frame>",
 "reason": "ID_FACE_DRIFT detected at t=1.2s"}

{"action": "set_camera_movement",
 "field": "camera_movement",
 "value": "<expected_camera_movement>",
 "reason": "CA_MOVE_WRONG detected in the localized span"}

{"action": "inject_prop",
 "field": "props_required",
 "value": "<missing_prop>",
 "reason": "AL_PROP_MISSING verified by object or VLM evidence"}

{"action": "regenerate_segment",
 "field": "segment",
 "value": "<t0,t1>",
 "reason": "MO_FROZEN_FRAME or MO_SEGMENT_BREAK is localized to this span"}
```

Each patch specifies an action type, the affected target, an optional update value, and a short rationale tied to the evidence span. When multiple records affect the same target, the compiler groups them before re-rendering to reduce conflicts among patch actions.

In the reported Human Pass@K evaluation, the concretely executed actions are the L2/L3 subset exposed by the active backend, primarily reference-frame injection, segment rerendering or temporal smoothing, enhancement or deblocking, and mask- or reference-conditioned prop insertion when supported. L0/L1 rows remain structured repair-plan instructions and are not counted as executable patches.

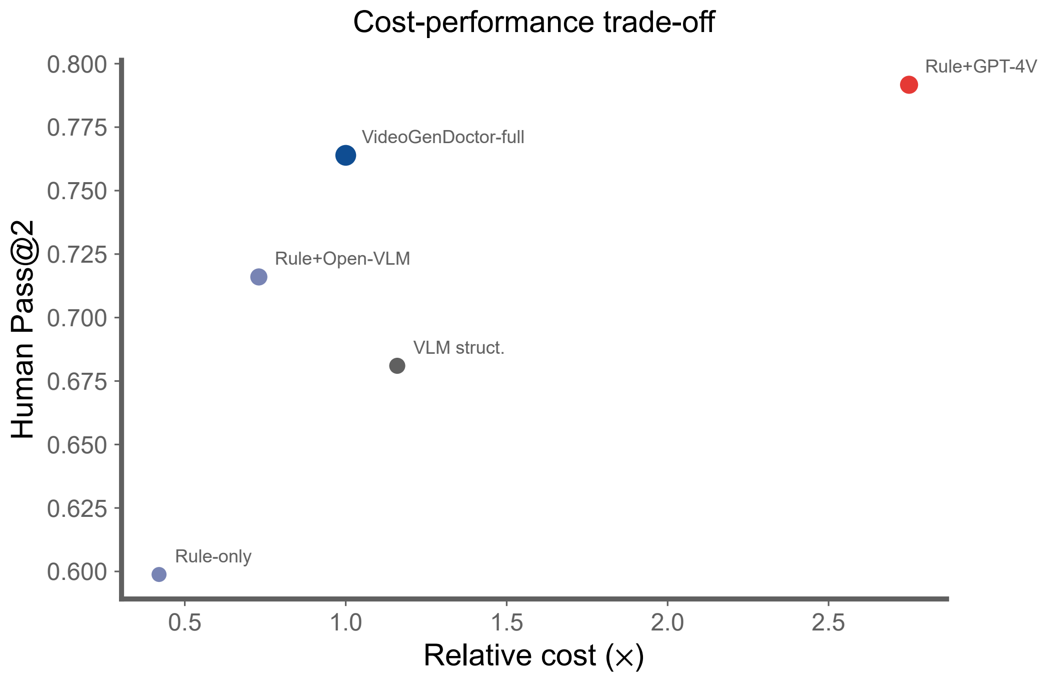

Figure 5: Pareto view of the cost-performance trade-off under the shared downstream repair protocol. The default VideoGenDoctor-full operating point improves human repair success over lower-cost rule variants and direct structured VLM reporting, while the higher-cost Rule+GPT-4V reference obtains the strongest Human Pass@2 at substantially higher relative cost.

Table 17: Adapter-executability stratification. L0: abstention; L1: prompt-level instructions; L2: partially executable actions; L3: native generator/editor controls.

Adapter level	Plan share	Adapter-executable rate	Human Pass@2	New artifacts
L0 abstain	0.100	0.000	0.280	0.040
L1 plan	0.310	0.280	0.540	0.100
L2 partial	0.420	0.740	0.730	0.140
L3 native	0.170	0.960	0.810	0.160

F Annotation guide summary

Annotators review each perturbed video together with the original prompt or ShotIR specification and the auto-assigned candidate failure codes. For each candidate, annotators determine whether the failure is visually present, assign the tightest temporal span that contains the visible deviation, and select representative keyframes when available. Annotators may also add missing failures if the perturbation produces an additional visible artifact.

The annotation record is stored as one JSON object per video and includes the video identifier, annotator identifier, verified failure codes, confidence values, evidence spans, and free-form notes. A failure is retained in the benchmark only when the annotator can verify it from the rendered video and specification, without relying on perturbation metadata alone. Ambiguous cases are handled conservatively: if a deviation is not visually identifiable, the candidate is marked as not verified.

The experiment fixture includes a dual-annotated reliability subset of 120 videos and 768 candidate evidence spans. The subset contains both controlled perturbation samples and naturally occurring real-failure samples. Larger multi-annotator validation with more than two independent annotators remains future work.

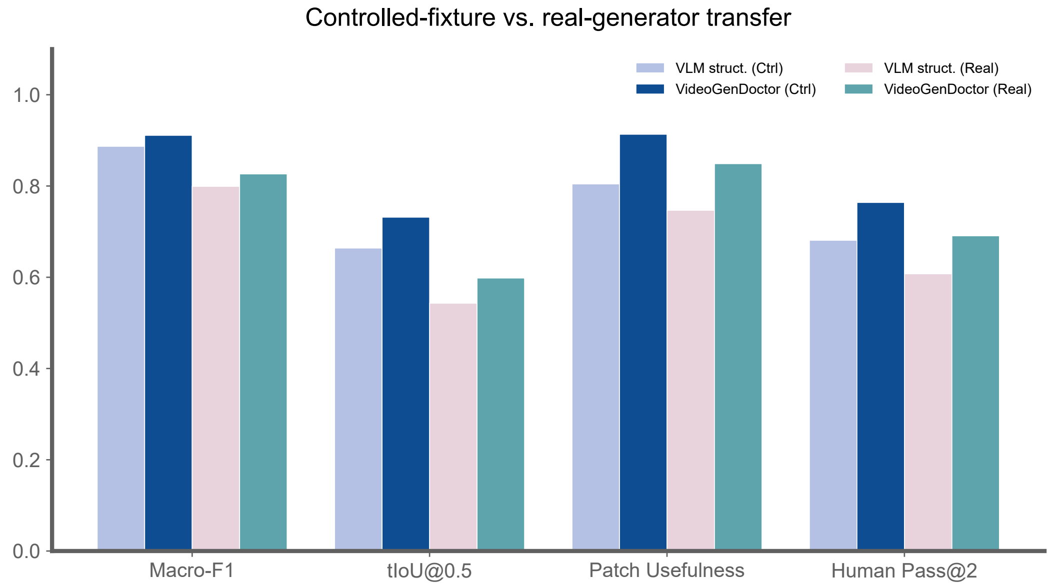

Figure 6: Controlled-to-real transfer across four metrics. Solid bars: controlled fixture; hatched bars: real-failure subset. The degradation gap is largest for tIoU@0.5 and smallest for patch usefulness.

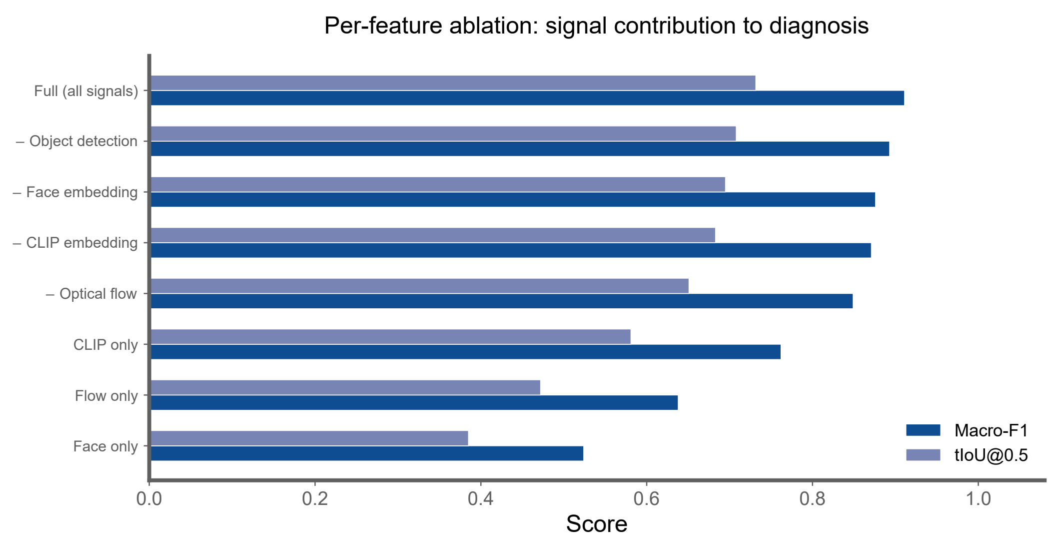

Figure 7: Per-feature ablation horizontal bar chart. Optical flow is the single most critical signal; single-signal configurations all underperform substantially.

G Example diagnostic report

H Direct VLM baseline prompt templates

The four direct VLM/LLM baselines differ only in their allowed information access and output schema. The free-form and structured VLM report baselines receive sampled frames and the specification. The VLM-to-patch baseline also receives sampled frames, the taxonomy names, and the repair-plan vocabulary, but not VideoGenDoctor's rule scores or patch compiler outputs. The Score+LLM-to-patch baseline receives scalar evaluator scores and the specification only; it does not receive sampled frames. These information boundaries are fixed before evaluation and are used for all samples.

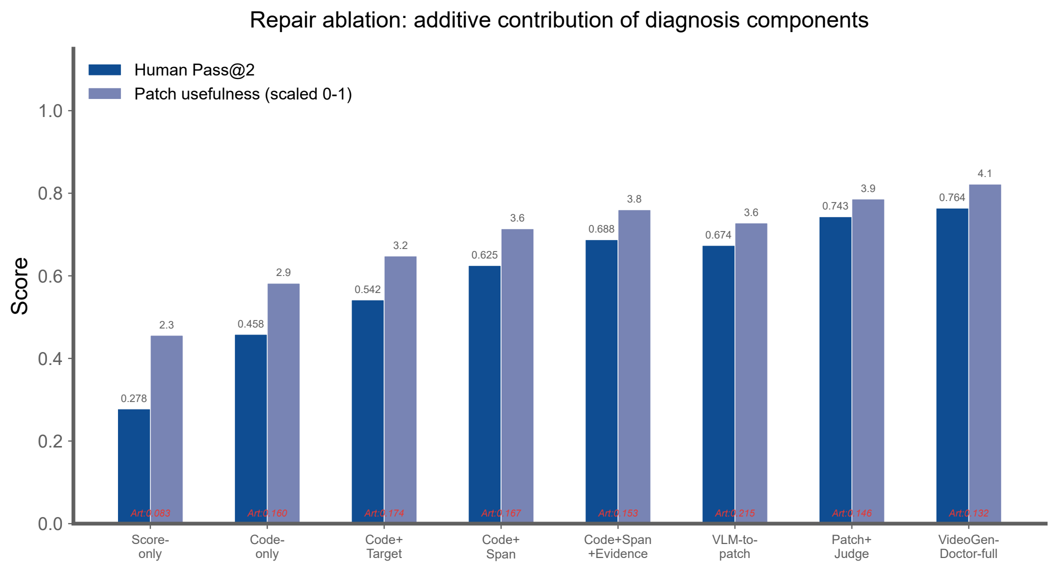

Figure 8: Repair ablation as grouped bars. Human Pass@2 and patch usefulness rise additively as code, span, evidence, and verification are incorporated.

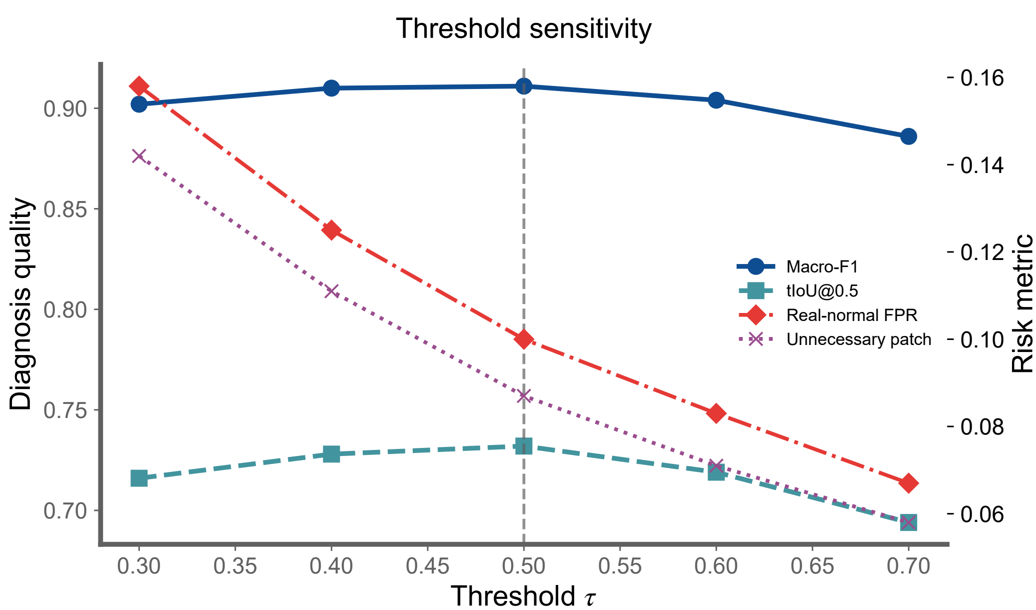

Figure 9: Threshold sensitivity curves. Dashed line at $\tau = 0.50$ marks the operating point where macro-F1 and tIoU@0.5 peak.

F.1 Free-form VLM report prompt. The model receives the video specification, sampled keyframes with timestamps, and a short instruction: identify visible failures, explain the evidence, and propose repair suggestions. The response is then parsed into taxonomy codes and evidence spans for evaluation.

F.2 Structured VLM report prompt. The model receives the same input but must output JSON objects with fields `failure_code`, `temporal_span`, `affected_target`, `confidence`, `evidence_keyframes`, `rationale`, and `repair_action`. Failures without visual evidence must be marked as uncertain rather than hallucinated.

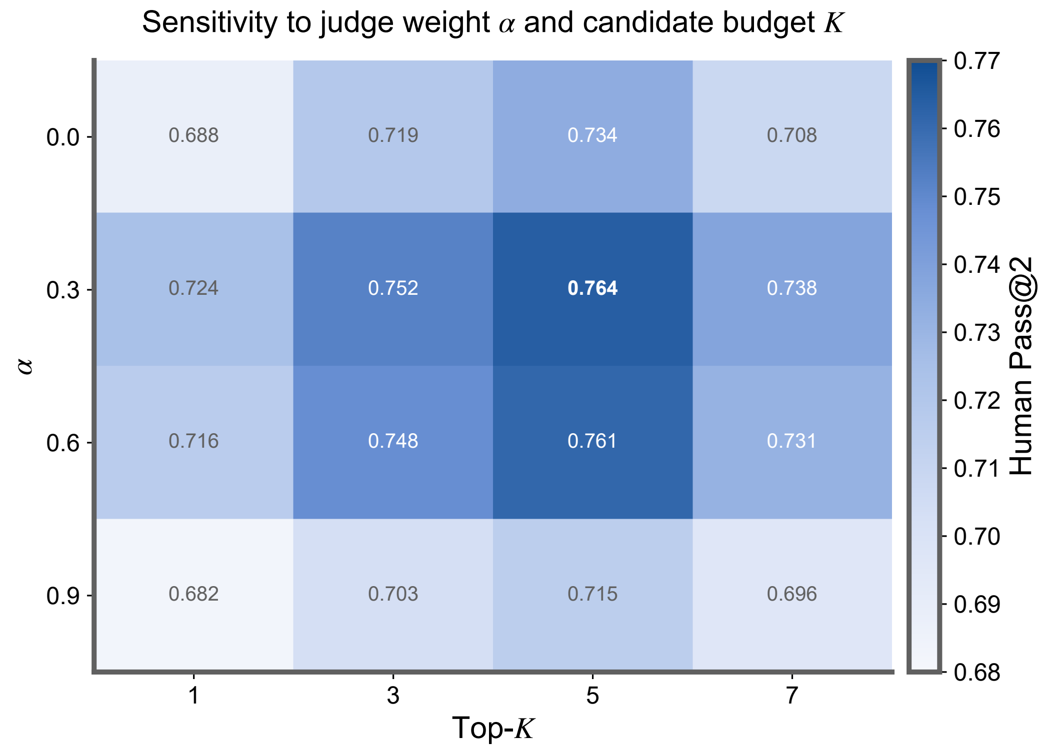

Figure 10: Sensitivity to judge weight α and candidate budget K. $\alpha = 0.6$, $K = 3$ balances diagnosis accuracy and repair success.

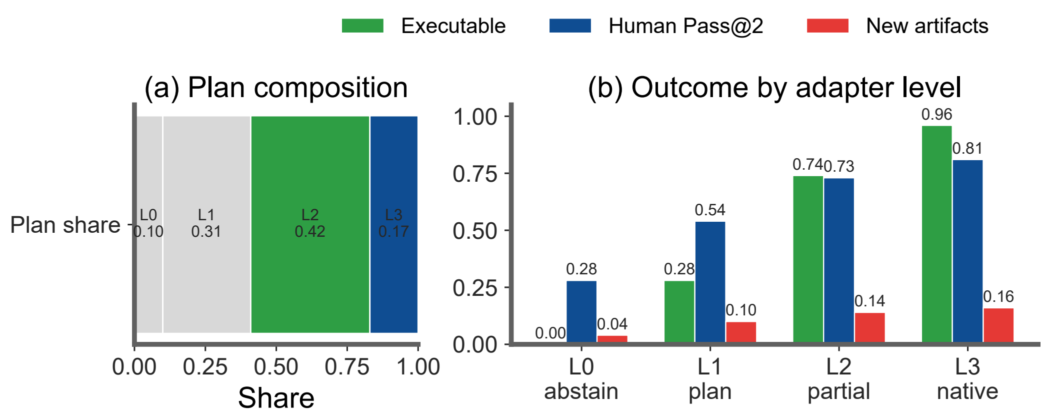

Figure 11: Adapter-executability stratification by repair-plan level. The left panel reports the share of generated plans by adapter level; the right panel compares adapter-executable rate, Human Pass@2, and new-artifact rate within each level.

```
You are evaluating whether a generated video follows the given specification.
For each visible failure, return a JSON record:
{
  "failure_code": "<taxonomy code>",
  "temporal_span": [t0, t1],
  "affected_target": "<character / prop / camera / motion / style / scene>",
  "confidence": <0-1>,
  "evidence_keyframes": ["frame@time", ...],
  "rationale": "short visual reason",
```

Table 18: Stress validation on harder real-failure slices. Intervals are bootstrap half-widths for Figure 12.

Slice	Macro-F1	tIoU@0.5	Human Pass@2	FPR
Base real-failure	0.826 ± 0.020	0.598 ± 0.017	0.691 ± 0.028	0.100 ± 0.010
New generators	0.802 ± 0.024	0.574 ± 0.020	0.662 ± 0.034	0.126 ± 0.012
Long videos	0.784 ± 0.029	0.552 ± 0.025	0.628 ± 0.041	0.142 ± 0.014
Style shift	0.811 ± 0.025	0.581 ± 0.022	0.674 ± 0.036	0.118 ± 0.013
Complex prompts	0.793 ± 0.027	0.560 ± 0.023	0.641 ± 0.039	0.135 ± 0.014

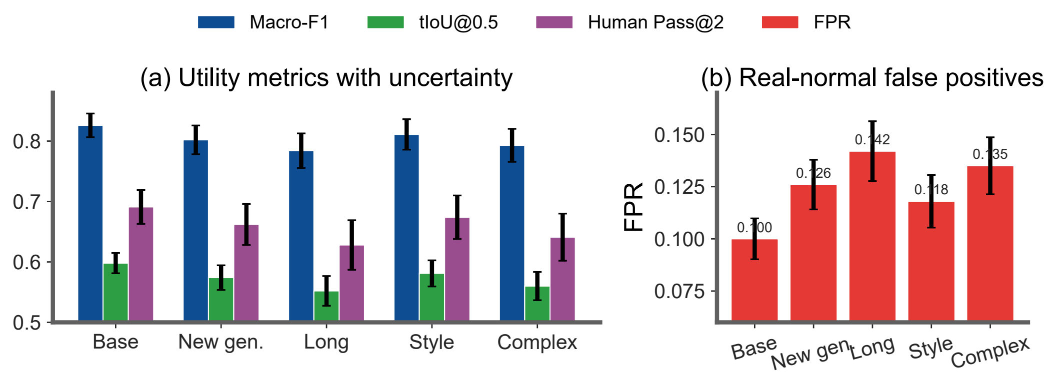

Figure 12: Stress validation on harder real-failure slices. Left: diagnosis, localization, and human repair with uncertainty. Right: false-positive rate on matched real-normal controls.

```
  "repair_action": "<patch action>"
}
Only report failures that are visually supported by the frames.
```

F.3 VLM-to-patch prompt. The VLM-to-patch baseline receives the same video specification and sampled keyframes as the direct VLM diagnosis baselines, but it is asked to output repair actions directly rather than diagnosis records. This baseline is allowed to access the taxonomy names and the supported repair-plan vocabulary, but it is not given VideoGenDoctor's Stage-1 rule scores, candidate segments, or patch compiler outputs. It may infer temporal spans from the sampled frames, but it does not receive ground-truth spans or VideoGenDoctor-predicted spans. This setting tests whether a VLM can directly produce structured repair plans from the specification and visual evidence.

```
You are a video-generation repair planner.

Input:
1. A video specification or ShotIR-style instruction.
2. Sampled keyframes from the generated video, each with a timestamp.
3. The failure-code taxonomy names.
4. The supported repair-plan vocabulary.

Your task:
Inspect the sampled frames and the specification. Identify only visually supported
problems and output structured repair-plan actions. Do not write a free-form critique.
Do not invent failures that are not visible in the frames.

You may use the following failure-code groups:
- Identity: ID_FACE_DRIFT, ID_BODY_DRIFT, ID_WRONG_CHARACTER,
  ID_CLOTHING_CHANGE, ID_MULTIPLE_FACES, ID_NO_FACE_VISIBLE
- Camera: CA_MOVE_WRONG, CA_SHOT_TYPE_WRONG, CA_UNINTENDED_ZOOM,
  CA_SHAKE, CA_WRONG_ANGLE, CA_ROLL
- Motion: MO_JITTER, MO_FRAME_DROP, MO_UNNATURAL_MOTION,
  MO_EVENT_MISSING, MO_ACTION_ORDER_WRONG, MO_MOTION_BLUR_EXCESS,
  MO_FROZEN_FRAME, MO_SEGMENT_BREAK
```

Table 19: Three-annotator stability analysis. Boundary disagreement in seconds (lower is better).

Subset	#Videos	#Candidate spans	Annotators	Fleiss' κ	Mean pairwise tIoU / Boundary disagreement
Controlled	72	468	3	0.842	0.704 / 0.38s
Real-failure	48	300	3	0.782	0.628 / 0.52s
Combined	120	768	3	0.813	0.666 / 0.45s

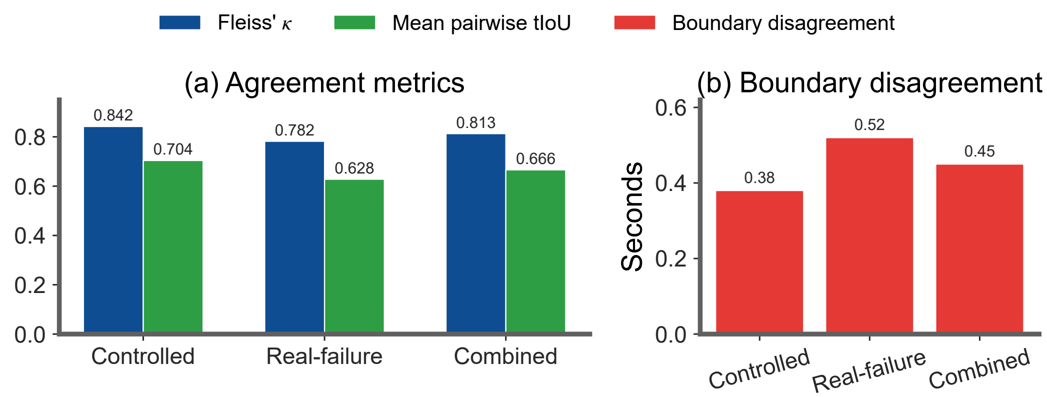

Figure 13: Multi-annotator stability. Code-level κ exceeds temporal-boundary agreement; the real-failure split shows highest ambiguity.

```
- Alignment: AL_PROP_MISSING, AL_PROP_WRONG_PLACEMENT,
  AL_TEXT_PROMPT_MISMATCH, AL_CHARACTER_COUNT_WRONG,
  AL_WRONG_PROP, AL_PROP_OCCLUDED
- Style: ST_COLOR_SHIFT, ST_ART_STYLE_INCONSISTENCY,
  ST_COMPRESSION_ARTIFACT, ST_TEXTURE_FLICKER,
  ST_OVEREXPOSURE, ST_UNDEREXPOSURE
- Scene: SC_BG_INCONSISTENCY, SC_ENVIRONMENT_MISMATCH,
  SC_LIGHTING_SHIFT, SC_OBJECT_TELEPORT, SC_BG_FLICKER,
  SC_WRONG_TIME_OF_DAY

Supported repair-plan vocabulary:
- inject_reference_frame
- replace_character_anchor
- fix_character_consistency
- adjust_framing
- set_camera_movement
- set_camera_angle
- stabilize_camera
- remove_zoom
- remove_camera_roll
- inject_prop
- reposition_prop
- remove_prop
- adjust_character_count
- strengthen_prompt_guidance
- inject_action_keyframe
- reorder_segments
- regenerate_segment
- interpolate_frames
- normalize_frame_rate
- constrain_motion
- smooth_segment_transition
- reduce_motion_blur
- apply_color_grading
```

Table 20: Operator-template alignment analysis for the controlled fixture. The table makes explicit which observable cues, rule signals, failure codes, and repair-plan templates are aligned by construction.

Perturbation operator	Observable cue	Rule / feature signal	Failure code	Repair-plan template	Alignment type
Identity-anchor removal	face or body appearance drift	face embedding drift; character-region embedding drift	`ID_FACE_DRIFT`, `ID_BODY_DRIFT`	`inject_reference_frame`; `replace_character_anchor`	identity-anchor
Required-prop dropping	required object becomes absent or inconsistent	object detector absence; VLM prop-absence judgment	`AL_PROP_MISSING`	`inject_prop`	prop-presence
Camera-movement alteration	requested trajectory is replaced by static or wrong motion	global flow direction; trajectory mismatch	`CA_MOVE_WRONG`	`set_camera_movement`	motion-control
Shot-type / framing alteration	framing violates requested shot type	crop statistics; VLM framing judgment	`CA_SHOT_TYPE_WRONG`	`adjust_framing`	framing-control
Camera-shake injection	unstable camera with unintended jitter	flow instability; camera-shake heuristic	`CA_SHAKE`	`stabilize_camera`	stability-control
Action dropping / shifting	required event missing or misordered	action-track gap; VLM event-order judgment	`MO_EVENT_MISSING`	`inject_action_keyframe`; `regenerate_segment`	event-structure
Temporal jitter / frame-drop injection	discontinuity, duplicate frames, or rate mismatch	flow anomaly; near-zero motion plateau	`MO_JITTER`, `MO_SEGMENT_BREAK`	`normalize_frame_rate`; `regenerate_segment`	temporal-continuity
Compression re-encoding	blockiness and deblocking artifacts	compression detector; texture degradation signal	`ST_COMPRESSION_ARTIFACT`	`apply_enhancement`	quality-restoration
Background-anchor weakening	scene background drifts across the shot	scene embedding drift; background-anchor mismatch	`SC_BG_INCONSISTENCY`	`fix_background_anchor`	scene-anchor

Table 21: Source composition of the failure-enriched real-generator subset. We generate 480 raw videos and retain 240 samples with annotator-verified visible failures. Sampling is balanced at 60 retained failures per generator, and generator-specific prompt or conditioning adapters are logged in the construction manifest. The subset is used only as an initial transfer check, not to estimate natural failure prevalence. We use publicly identifiable generator checkpoints to make the subset construction auditable and reproducible.

Generator	Version / checkpoint	Input type	#Prompts / specs	#Videos	Avg. duration	#Unique observed codes
CogVideoX	CogVideoX1.5-5B	text / ShotIR-to-prompt	60	60	8.4s	15
Wan	Wan2.1-T2V-14B	text / ShotIR-to-prompt	60	60	9.2s	16
Stable Video Diffusion	SVD-XT-1.1	image-to-video / reference frame	60	60	8.6s	14
HunyuanVideo	HunyuanVideo-1.5	text / multi-shot prompt	60	60	9.5s	17
Total	–	–	240	240	8.93s	18

```
- apply_style_transfer
- apply_enhancement
- smooth_texture
- adjust_exposure
- fix_background_anchor
- set_environment
- normalize_lighting
- stabilize_scene_objects
- set_time_of_day
- no_op_uncertain

Return a JSON object with the following schema:
{
  "patches": [
    {
      "patch_id": "<unique patch id>",
      "failure_code": "<taxonomy code or uncertain>",
      "action": "<one item from the supported repair-plan vocabulary>",
      "target": "<character / prop / camera / motion / style / scene target>",
      "temporal_span": [<start_sec>, <end_sec>],
      "evidence_keyframes": ["frame@time", "..."],
      "confidence": <0-1>,
      "patch_fields": {
        "field": "<generator or ShotIR field to modify>",
        "value": "<new value, constraint, or reference to insert>"
```

Table 22: Real-failure and real-normal evaluation. The real-normal subset contains 120 videos retained from the same 480 raw generations but judged to contain no obvious failure. Whenever a diagnosis-oriented front end is listed, patch-related columns are computed after pairing that front end with the shared downstream repair loop. Best values are boldfaced; higher is better except for real-normal FPR and unnecessary patch rate, where lower is better.

Method	Real-failure Recall	Real-normal FPR	Unnecessary Patch Rate	No-op Accuracy	Specificity
Score-only	0.621	0.258	0.231	0.769	0.742
Rule-only	0.806	0.168	0.154	0.846	0.832
VLM structured report	0.818	0.217	0.205	0.795	0.783
VLM structured + self-consistency	0.837	0.167	0.150	0.850	0.833
VLM temporal proposal + patch	0.848	0.192	0.183	0.817	0.808
VideoGenDoctor-diagnosis	**0.862**	0.108	0.092	0.908	0.892
VideoGenDoctor-full	0.858	**0.100**	**0.087**	**0.913**	**0.900**

Table 23: Stress analysis on 72 ambiguous specification-violation cases. These samples contain possible but non-adjudicable deviations from the specification. Whenever a diagnosis-oriented front end is listed, patch-related columns are computed after pairing that front end with the shared downstream repair loop. Best values are boldfaced; higher is better for abstention-oriented columns, lower is better for false concrete patch, wrong-target patch, over-repair, and new artifacts.

Method	Abstention rate	Appropriate abstention	False concrete patch	Wrong-target patch	Over-repair	New artifacts
Score-only	0.292	0.361	0.514	0.222	0.347	0.083
VLM structured report	0.181	0.292	0.611	0.278	0.431	0.153
VLM temporal proposal + patch	0.222	0.347	0.556	0.236	0.389	0.194
VideoGenDoctor-diagnosis	**0.500**	**0.639**	0.292	0.125	0.208	**0.069**
VideoGenDoctor-full	0.472	0.625	**0.250**	**0.111**	**0.181**	0.111

```
      },
      "rationale": "<short visual reason>"
    }
  ],
  "global_notes": "<one-sentence summary>",
  "uncertain": <true or false>
}

Rules:
1. Output valid JSON only.
2. Every patch must use an action from the supported repair-plan vocabulary.
3. If no visually supported failure is visible, return:
   {"patches": [], "global_notes": "No visually supported repair action.", "uncertain": true}
4. If the temporal span is uncertain, estimate it from the nearest sampled frame
   timestamps and set "confidence" below 0.5.
5. Do not output patches for purely aesthetic preferences unless the specification
   explicitly requires them.
```

F.4 Score+LLM-to-patch prompt. The Score+LLM-to-patch baseline receives scalar evaluator outputs and the original video specification, but it does not receive sampled frames. This baseline is intentionally weaker in visual grounding but represents a common production setting where low-level evaluators return scalar scores and an LLM converts low-scoring dimensions into repair suggestions. Because it cannot inspect frames, it is not allowed to claim frame-specific visual evidence. It may output approximate temporal spans only when the scalar evaluator provides segment-level scores; otherwise the temporal span must be marked as `null`. This makes the information boundary explicit and prevents unfair comparison with frame-based VLM baselines.

```
You are a repair planner for controllable video generation.

Input:
1. The original video specification or ShotIR-style instruction.
2. Scalar evaluator outputs, such as quality, alignment, motion, identity,
   camera, style, and scene-consistency scores.
3. Optional segment-level scalar scores if available.
4. The supported repair-plan vocabulary.
```

Table 24: Stress analysis on 48 low-quality or non-adjudicable cases. These samples are dominated by global degradation, unclear content, or insufficient evidence for reliable code-level labels. Whenever a diagnosis-oriented front end is listed, patch-related columns are computed after pairing that front end with the shared downstream repair loop. Best values are boldfaced; higher is better for quality-gated abstention, lower is better for the remaining columns.

Method	Quality-gated abstention	False concrete patch	Wrong-target patch	Over-repair	New artifacts
Score-only	0.417	0.375	0.125	0.208	0.063
VLM structured report	0.292	0.458	0.188	0.271	0.104
VLM temporal proposal + patch	0.333	0.396	0.167	0.250	0.146
VideoGenDoctor-diagnosis	**0.667**	0.146	0.063	0.104	**0.042**
VideoGenDoctor-full	0.625	**0.125**	**0.042**	**0.083**	0.083

Table 25: Observed failure-code distribution in the controlled fixture and real-failure subset. The full taxonomy contains 38 codes, but empirical claims are restricted to the observed codes below.

Failure code	Group	Controlled spans	Real-failure spans	Total spans
`ID_FACE_DRIFT`	Identity	150	112	262
`ID_BODY_DRIFT`	Identity	132	76	208
`ID_CLOTHING_CHANGE`	Identity	–	60	60
`CA_MOVE_WRONG`	Camera	240	104	344
`CA_SHOT_TYPE_WRONG`	Camera	144	84	228
`CA_SHAKE`	Camera	126	70	196
`CA_WRONG_ANGLE`	Camera	–	68	68
`MO_JITTER`	Motion	174	124	298
`MO_FROZEN_FRAME`	Motion	156	72	228
`MO_SEGMENT_BREAK`	Motion	162	88	250
`MO_EVENT_MISSING`	Motion	126	90	216
`MO_ACTION_ORDER_WRONG`	Motion	–	56	56
`AL_PROP_MISSING`	Alignment	198	116	314
`AL_TEXT_PROMPT_MISMATCH`	Alignment	–	132	132
`ST_COMPRESSION_ARTIFACT`	Style	252	80	332
`ST_COLOR_SHIFT`	Style	–	56	56
`SC_BG_INCONSISTENCY`	Scene	156	86	242
`SC_OBJECT_TELEPORT`	Scene	–	62	62
Total	–	2016	1536	3552

```
Important information boundary:
You do NOT receive sampled frames or the video itself. Therefore:
- Do not claim that a failure is visually observed.
- Do not mention specific visual evidence or keyframes.
- Only infer likely repair actions from low-scoring evaluator dimensions.
- Use "evidence_source": "scalar_score" for all patches.
- Use "temporal_span": null unless segment-level scores identify a specific interval.

Supported repair-plan vocabulary:
- inject_reference_frame
- replace_character_anchor
- fix_character_consistency
- adjust_framing
- set_camera_movement
- set_camera_angle
- stabilize_camera
- remove_zoom
- remove_camera_roll
- inject_prop
- reposition_prop
- remove_prop
- adjust_character_count
- strengthen_prompt_guidance
- inject_action_keyframe
- reorder_segments
- regenerate_segment
```

Table 26: Annotation reliability. Dual annotation is performed on a reliability subset before adjudication.

Subset	#Videos	#Candidate spans	#Annotators	Code agreement	Span agreement
Controlled reliability subset	72	468	2	$\kappa = 0.858$	tIoU = 0.716
Real-failure reliability subset	48	300	2	$\kappa = 0.803$	tIoU = 0.641
Combined	120	768	2	$\kappa = 0.831$	tIoU = 0.681

Table 27: Implementation details for VLM-based judges and external diagnosis/repair baselines.

Setting	Model	Frames	Res.	Prompt	Rel. cost
Rule+Open-VLM	Qwen2-VL-7B-Instruct Wang et al. [2024]	18	512px short side	structured QA	0.34×
Rule+GPT-4V	GPT-4V endpoint	18	512px short side	structured QA	2.75×
VLM free-form report	GPT-4V endpoint	12	512px short side	free-form critique	1.12×
VLM structured report	GPT-4V endpoint	12	512px short side	JSON schema	1.16×
VLM structured + self-consistency	GPT-4V endpoint	12 frames × 3 runs	512px short side	JSON schema, 3 runs	3.48×
VLM temporal proposal + patch	GPT-4V endpoint	12	512px short side	span proposal + JSON	2.05×
VLM-to-patch	GPT-4V endpoint	12	512px short side	JSON patch	1.19×
Score+LLM-to-patch	scalar evaluator + LLM	0	–	score-to-patch JSON	N/A

Note. Most direct VLM baselines share the same hosted GPT-4V endpoint OpenAI [2023] to isolate prompt- and schema-level effects rather than backbone changes; Rule+Open-VLM provides an open-model reference. Hosted endpoint calls are logged with raw responses for auditability. 'N/A' denotes a scalar-evaluator-dominated pipeline whose cost is not directly comparable to frame-based VLM calls.

```
- interpolate_frames
- normalize_frame_rate
- constrain_motion
- smooth_segment_transition
- reduce_motion_blur
- apply_color_grading
- apply_style_transfer
- apply_enhancement
- smooth_texture
- adjust_exposure
- fix_background_anchor
- set_environment
- normalize_lighting
- stabilize_scene_objects
- set_time_of_day
- no_op_uncertain

Return a JSON object with the following schema:
{
  "patches": [
    {
      "patch_id": "<unique patch id>",
      "score_dimension": "<low-scoring evaluator dimension>",
      "inferred_failure_type": "<short inferred failure description>",
      "action": "<one item from the supported repair-plan vocabulary>",
      "target": "<character / prop / camera / motion / style / scene target>",
      "temporal_span": null,
      "evidence_source": "scalar_score",
      "triggering_scores": {
        "<score_name>": <score_value>
      },
      "confidence": <0-1>,
      "patch_fields": {
        "field": "<generator or ShotIR field to modify>",
        "value": "<new value, constraint, or reference to insert>"
      },
      "rationale": "<short reason based only on scalar scores and specification>"
```

Table 28: Per-code reliability on the real-failure subset. The subset is more challenging than the controlled fixture, especially for motion, camera, and scene-level failures.

Failure code	#Spans	Precision	Recall	F1	tIoU@0.5
`ID_FACE_DRIFT`	112	0.884	0.848	0.866	0.648
`ID_BODY_DRIFT`	76	0.856	0.823	0.839	0.615
`ID_CLOTHING_CHANGE`	60	0.834	0.798	0.816	0.586
`CA_MOVE_WRONG`	104	0.866	0.832	0.849	0.628
`CA_SHOT_TYPE_WRONG`	84	0.838	0.803	0.820	0.590
`CA_SHAKE`	70	0.820	0.779	0.799	0.559
`CA_WRONG_ANGLE`	68	0.825	0.786	0.805	0.568
`MO_JITTER`	124	0.823	0.777	0.799	0.565
`MO_FROZEN_FRAME`	72	0.899	0.854	0.876	0.663
`MO_SEGMENT_BREAK`	88	0.827	0.792	0.809	0.582
`MO_EVENT_MISSING`	90	0.807	0.762	0.784	0.542
`MO_ACTION_ORDER_WRONG`	56	0.792	0.748	0.769	0.524
`AL_PROP_MISSING`	116	0.893	0.863	0.878	0.669
`AL_TEXT_PROMPT_MISMATCH`	132	0.842	0.794	0.817	0.596
`ST_COMPRESSION_ARTIFACT`	80	0.914	0.874	0.893	0.688
`ST_COLOR_SHIFT`	56	0.846	0.806	0.825	0.601
`SC_BG_INCONSISTENCY`	86	0.832	0.795	0.813	0.578
`SC_OBJECT_TELEPORT`	62	0.823	0.782	0.802	0.545

Table 29: Per-generator results on the real-failure and real-normal subsets. FPR is computed on 30 real-normal samples per generator.

Generator	Macro-F1	tIoU@0.5	Human Pass@2	Real-normal FPR	Main failure groups	Normal samples
CogVideoX	0.834	0.609	0.702	0.092	Camera, Motion, Alignment	30
Wan	0.821	0.588	0.681	0.108	Motion, Camera, Identity	30
Stable Video Diffusion	0.842	0.617	0.696	0.083	Identity, Style, Scene	30
HunyuanVideo	0.809	0.579	0.684	0.117	Camera, Motion, Alignment	30
Average	0.827	0.598	0.691	0.100	–	120

```
    }
  ],
  "global_notes": "<one-sentence summary>",
  "uncertain": <true or false>
}

Rules:
1. Output valid JSON only.
2. Do not claim visual evidence because frames are not provided.
3. Do not output frame IDs or evidence keyframes.
4. Use temporal_span = null unless segment-level scalar scores are provided.
5. Every patch must use an action from the supported repair-plan vocabulary.
6. If the scalar scores do not indicate a clear repair target, return:
   {"patches": [], "global_notes": "No reliable score-based repair action.", "uncertain": true}
```

F.5 Parsing and invalid-output handling. Invalid JSON outputs are repaired using a deterministic parser when possible. If the output cannot be parsed, the prediction is marked invalid. Failure codes outside the taxonomy are mapped to the nearest taxonomy code only when an exact synonym exists; otherwise they are counted as false positives. Predictions without temporal spans receive no localization credit. Hallucinated failures without visual evidence are counted as false positives. Repair actions outside the supported repair-plan vocabulary are marked invalid for repair-plan scoring and non-adapter-executable for adapter scoring.

I Blind human repair evaluation protocol

Raters are shown the specification and one output video, without method names or diagnosis records. They answer whether the video satisfies the specification overall, whether the target failure is resolved, whether the repair introduces new artifacts, and how useful the patch is on a 1–5 scale. Human Pass@K is the fraction of samples judged successful after at most K repair attempts.

Table 30: Leave-one-perturbation-out evaluation. Each held-out operator is excluded from threshold tuning and then used only for testing.

Held-out perturbation operator	Macro-F1	tIoU@0.5	Top-3 hit
Identity-anchor removal	0.884	0.681	0.644
Required-prop dropping	0.902	0.704	0.672
Camera-movement alteration	0.871	0.653	0.612
Shot-type / framing alteration	0.862	0.641	0.606
Camera-shake injection	0.851	0.628	0.590
Action dropping / shifting	0.838	0.612	0.568
Temporal jitter / frame-drop injection	0.846	0.621	0.577
Compression re-encoding	0.910	0.719	0.688
Background-anchor weakening	0.868	0.646	0.610
Average	0.870	0.656	0.619

Table 31: Leave-one-source-group-out evaluation over the six controlled-fixture domain groups.

Held-out source group	Macro-F1	tIoU@0.5	Top-3 hit
Indoor / human-object	0.899	0.714	0.665
Outdoor / motion	0.887	0.702	0.651
Multi-object interaction	0.892	0.706	0.657
Camera-control scene	0.875	0.683	0.631
Style / lighting variation	0.907	0.722	0.676
Scene-change / background	0.884	0.694	0.642
Average	0.891	0.704	0.654

A repaired video is marked as pass if the target failure is resolved and no severe new artifact is introduced. Human Pass@K is the fraction of samples judged successful after at most K repair attempts. New artifacts are marked when raters observe a visible degradation introduced by repair that was not present in the original generated video. Each repaired video is rated by 3 independent raters. We report the majority-vote pass/fail label, the mean patch-usefulness score, and the majority-vote new-artifact label.

J Bootstrap confidence intervals

We use 10,000 video-level bootstrap resamples and report percentile 95% confidence intervals. For the controlled fixture, resampling is performed over 324 videos. For the real-failure subset, resampling is performed over 240 videos. For blind human repair evaluation, resampling is performed over the stratified human-rated subset. For paired comparisons, we resample videos with replacement and compute the distribution of metric differences between two methods. Small point-estimate differences are not treated as meaningful unless the paired confidence interval excludes zero.

The large degradation under leave-one-code-family-out ablation confirms that VideoGenDoctor should not be interpreted as an unconstrained open-ended failure discovery system. The experiment clarifies which portions of the performance depend on family-specific detectors and which portions remain supported by generic evidence signals and VLM verification.

K Real-failure construction manifest

When a generator does not expose one of these configuration fields, the corresponding manifest value is set to `null` rather than omitted. For hosted or version-changing endpoints, we log the endpoint identifier, access date, raw request payload, and raw response metadata.

To make the real-failure subset auditable and reproducible, each generated video is accompanied by a JSON manifest. The manifest records the generator identity, checkpoint, input specification, generation configuration, output location, screening decision, verified failure codes, temporal evidence spans, and annotator identifiers. This protocol turns the real-failure subset from an informal collection of generated examples into a traceable evaluation resource.

Each manifest entry contains the following fields: `generator`, `checkpoint`, `prompt_id`, `input_type`, `shotir_spec`, `shotir_to_prompt_converted_text`, `reference_frame_id`,

Table 33: Patch-to-generator adapter details. A repair plan is counted as adapter-executable only when it can be automatically translated into target-generator or editor inputs without manual rewriting. L0/L1 rows denote abstention or prompt-rewrite instructions rather than executable patches.

Repair-plan action	ShotIR field	Counted as	Adapter availability	Concrete generator/editor input	Fallback
inject_reference_frame	character.identity_anchor	L2	L2 available for SVD; L1 partial for text-to-video models	reference image input + identity prompt strengthening	prompt-only repair plan
set_camera_movement	camera.movement	L1	L1 partial	camera-control prompt rewrite; trajectory adapter when supported	no_op_uncertain if no camera control exists
set_camera_angle	camera.viewpoint	L1	L1 partial	viewpoint prompt rewrite; optional camera-control token	repair suggestion only
inject_prop	props_required	L1 / L2	L1 available; L2 partial with mask/reference input	prop phrase insertion + optional reference or mask	prompt edit
reposition_prop	props.region	L1	L1 partial	region-aware prompt rewrite or mask-conditioned edit	prompt-only repair plan
regenerate_segment	temporal span	L2 / L3	L2 available when segment rerendering is supported	segment-level rerender or temporal inpainting call	full-clip rerender
stabilize_camera	camera.constraint	L1 / L2	L1 partial	no-shake constraint or stabilization post-process	repair suggestion only
normalize_frame_rate	motion.continuity	L2	L2 partial	frame interpolation or temporal smoothing post-process	regenerate_segment
apply_enhancement	style.quality	L2	L2 available	enhancement / deblocking post-process	prompt edit for higher quality
fix_background_anchor	scene.background_anchor	L1	L1 partial	background-anchor prompt strengthening; optional reference image	prompt-only repair plan
no_op_uncertain	–	L0	L0 available	abstain from concrete repair	no action

Table 34: Blind human repair evaluation rubric for patch usefulness.

Score	Criterion
1	The patch does not address the target failure.
2	The patch is related to the failure but is not actionable or is largely insufficient.
3	The patch is actionable but incomplete or requires substantial manual interpretation.
4	The patch directly addresses the target failure with minor ambiguity.
5	The patch directly resolves the target failure and preserves surrounding valid content.

`seed`, `resolution`, `fps`, `num_frames`, `sampling_steps`, `guidance_scale`, `output_path`, `screening_status`, `verified_codes`, `evidence_spans`, and `annotator_ids`. The `screening_status` field takes one of four values: `generated`, `retained_candidate`, `verified`, or `excluded_ambiguous`. Only entries marked as `verified` are included in the reported real-failure evaluation.


```
{
  "video_id": "real_wan_00037",
  "generator": "Wan",
  "checkpoint": "Wan2.1-T2V-14B",
  "prompt_id": "shotir_prompt_037",
  "input_type": "text / ShotIR-to-prompt",
  "shotir_spec": {
    "shot_id": "S037",
    "duration_sec": 8.0,
    "characters": [
      {
        "id": "person_A",
        "description": "young man wearing a blue hoodie",
        "identity_anchor": "ref_person_A_004"
      }
    ],
    "required_props": [
      {
        "id": "red_water_bottle",
        "description": "red water bottle held in the right hand",
```

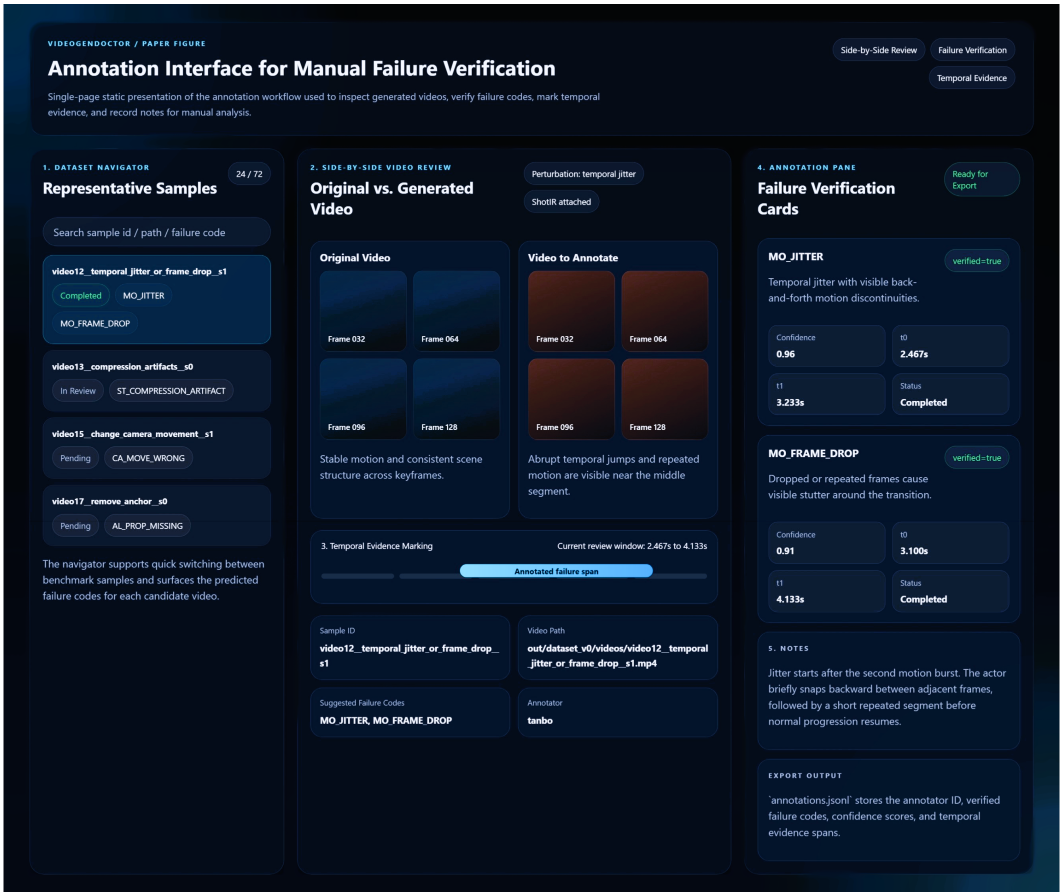

Figure 14: Annotation interface used for manual verification in VideoGenDoctor-Bench-v0. The tool exposes a sample navigator, side-by-side review of the original and perturbed videos, temporal evidence marking, structured failure verification cards, and free-form notes for adjudication.

Table 35: Error analysis on ambiguous and real-normal samples. The table focuses on no-op behavior and wrong-target repair planning. Best values are boldfaced for interpretable comparison columns; the no_op_uncertain rate is reported descriptively rather than bold-ranked.

Method	no_op_uncertain rate	Correct no-op rate	Missed necessary repair	Wrong-target patch	Over-confident patch
Score-only	0.286	0.524	0.119	0.171	0.321
VLM structured report	0.219	0.476	**0.096**	0.229	0.396
VLM temporal proposal + patch	0.257	0.518	0.108	0.200	0.350
VideoGenDoctor-diagnosis	0.552	**0.738**	0.142	0.095	0.181
VideoGenDoctor-full	0.514	0.724	0.156	**0.081**	**0.164**

```
      "expected_presence": "visible throughout the shot"
    }
  ],
  "camera": {
    "shot_type": "medium shot",
    "movement": "slow left-to-right tracking",
    "viewpoint": "front three-quarter view"
  },
  "action_order": [
    "person_A enters the park path",
    "person_A jogs while holding the red water bottle",
    "person_A slows down near the bench"
  ],
  "style": {
```

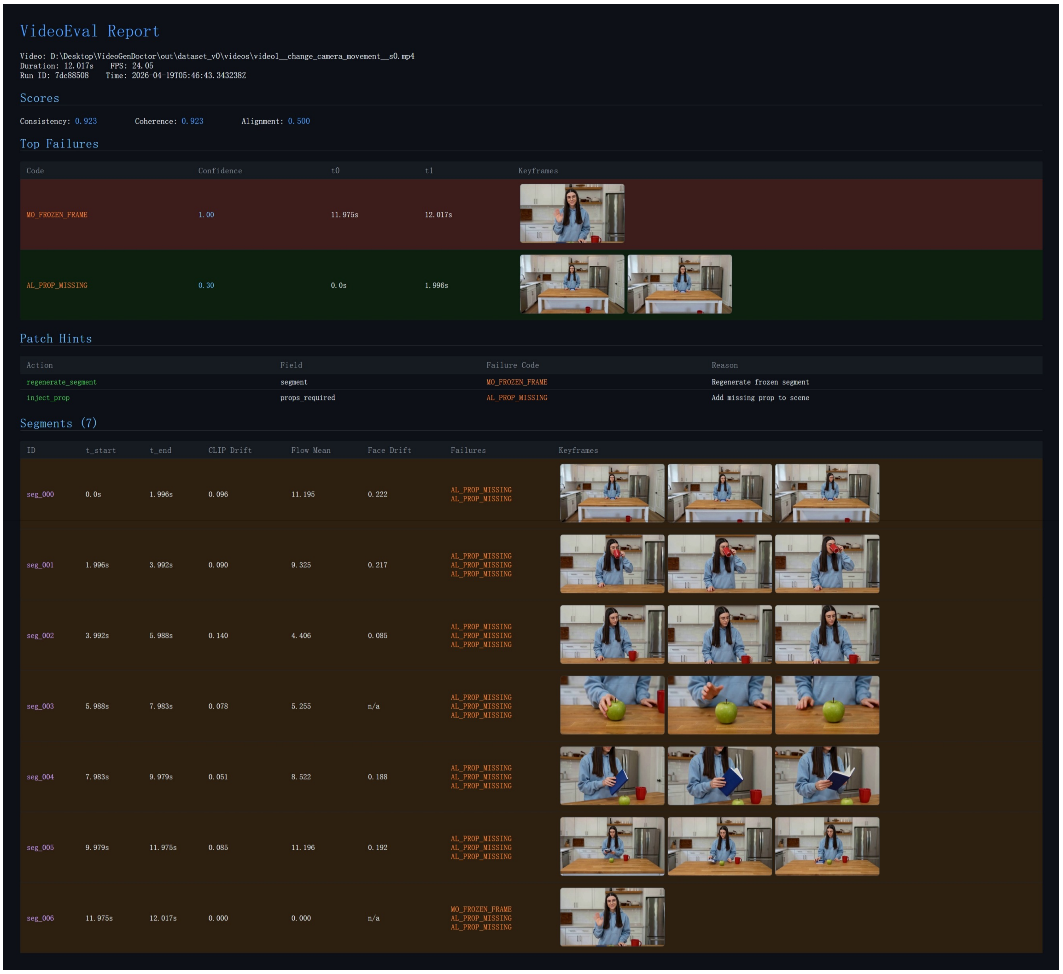

Figure 15: Example VideoGenDoctor diagnostic report for a perturbed video. The report summarizes global scores, ranked failure records with confidence and temporal spans, representative keyframes, patch hints derived from the retained failure codes, and segment-level evidence used for localized review and repair.

Table 36: Qualitative case-study summary. Cases are sampled from controlled, real-failure, and ambiguous stress subsets.

Case type	#Cases	Majority-judged successful / appropriate	Typical outcome
Successful structured repair plan	24	18	localized repair plan resolves target failure
Ambiguous case with abstention	24	16	no_op_uncertain avoids over-repair
Over-repair / wrong-target failure	24	7	concrete patch targets wrong object or span


```
      "lighting": "natural daylight",
      "visual_style": "realistic video"
    },
    "scene": {
      "location": "urban park path",
      "background_anchor": "trees and bench remain stable"
    }
  },
  "shotir_to_prompt_converted_text": "A realistic 8-second medium shot of a young man wearing a l
  "reference_frame_id": null,
  "seed": 18473,
  "resolution": "768x432",
  "fps": 16,
```

Table 37: Leave-one-code-family-out rule ablation. For the held-out family, dedicated family-specific rules are disabled; only generic embedding drift, optical-flow anomaly, scene-change peaks, and structured VLM verification are retained.

Held-out family	Macro-F1	tIoU@0.5	Top-3 hit	Main degradation source
Identity	0.782	0.579	0.536	face/body-specific anchors removed
Camera	0.754	0.546	0.501	camera-flow templates disabled
Motion	0.721	0.512	0.468	jitter / segment-break rules disabled
Alignment	0.801	0.603	0.558	prop detector and placement rules disabled
Style	0.836	0.641	0.590	compression/style detectors disabled
Scene	0.748	0.528	0.487	background-anchor rules disabled
Average	0.774	0.568	0.523	–

Table 38: Generator configuration metadata for the real-failure and real-normal subsets. Hosted or version-changing endpoints are logged with access date and endpoint identifier.

Generator	Checkpoint / endpoint	fps	Frames	Resolution	Steps	Guidance	Scheduler	Seed range	Reference source / prompt rule
CogVideoX	CogVideoX1.5-5B	16	128	768×432	50	6.0	DPMSolver	10000–10059	ShotIR-to-prompt template v1
Wan	Wan2.1-T2V-14B	16	128	768×432	50	5.0	FlowMatchEuler	18000–18059	ShotIR-to-prompt template v1
Stable Video Diffusion	SVD-XT-1.1	14	112	768×432	40	7.5	EulerDiscrete	26000–26059	reference frame from ShotIR anchor
HunyuanVideo	HunyuanVideo-1.5	16	128	768×432	50	7.0	FlowMatchEuler	34000–34059	multi-shot prompt template v1


```
  "num_frames": 128,
  "sampling_steps": 50,
  "guidance_scale": 7.5,
  "output_path": "real_failures/wan/real_wan_00037.mp4",
  "screening_status": "verified",
  "verified_codes": [
    "AL_PROP_MISSING",
    "CA_MOVE_WRONG",
    "MO_SEGMENT_BREAK"
  ],
  "evidence_spans": [
    {
      "failure_code": "AL_PROP_MISSING",
      "target": "red_water_bottle",
      "start_sec": 3.25,
      "end_sec": 6.75,
      "evidence_keyframes": [
        "frame_052@3.25s",
        "frame_080@5.00s",
        "frame_108@6.75s"
      ],
      "annotator_confidence": 0.91,
      "notes": "The specified red water bottle is not visible after the character begins jogging
    },
    {
      "failure_code": "CA_MOVE_WRONG",
      "target": "camera_movement",
      "start_sec": 0.00,
      "end_sec": 4.00,
      "evidence_keyframes": [
        "frame_000@0.00s",
        "frame_032@2.00s",
        "frame_064@4.00s"
      ],
      "annotator_confidence": 0.84,
      "notes": "The camera remains mostly static rather than performing the requested left-to-ri
    },
    {
      "failure_code": "MO_SEGMENT_BREAK",
```

```
      "target": "temporal_continuity",
      "start_sec": 5.50,
      "end_sec": 5.94,
      "evidence_keyframes": [
        "frame_088@5.50s",
        "frame_095@5.94s"
      ],
      "annotator_confidence": 0.78,
      "notes": "A visible discontinuity occurs near the transition to the final action."
    }
  ],
  "annotator_ids": [
    "ann_02",
    "ann_05"
  ]
}
```


For privacy and release safety, file paths may be anonymized, but the manifest must preserve all fields required to reproduce the generation configuration, reconstruct the ShotIR-to-prompt conversion, and audit the verified evidence spans. When a generator does not expose a configuration field such as `guidance_scale`, the field is set to `null` rather than omitted.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction (Section 1) explicitly state the scope—a controlled diagnostic fixture, not an open-world benchmark—and the four contributions match the experimental results reported in Sections 4.3–4.7. All claims are bounded by the limitations discussed in Section 5.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 5 includes a dedicated "Limitations" paragraph that discusses the controlled-fixture scope, failure-enriched sampling, verifier dependency, partial taxonomy coverage, annotation scale, and generator/domain/length coverage gaps. The "When does it fail?" paragraph further analyzes specific failure modes.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: This paper proposes a diagnosis-and-repair framework evaluated empirically. It does not contain theoretical results, theorems, or proofs.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The acknowledgments section describes the release package (reference implementation, taxonomy, patch compiler, generation scripts, annotations, evaluation scripts, prompt templates). The main experimental setup is detailed in Section 4.1 and Appendix B, including dataset splits, metrics, and evaluation protocols.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The reference implementation and all generated data (VideoGenDoctor-Bench-v0, annotations, prediction logs) will be released via GitHub under an MIT license upon acceptance. An anonymized repository is included in the supplementary material. Reproduction requires only CPU for table recomputation; GPU is needed only for full feature extraction and VLM judging.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4.1 and Appendix B specify the controlled-fixture construction (18 clips, 6 domain groups, 9 perturbation operators, 2 seeds), the real-failure sampling protocol (4 generators, 60 failures each, 480 raw $\rightarrow$ 240 retained), feature extractors (OpenCLIP, RAFT/Farneback, InsightFace, YOLOv8), default hyperparameters ($K = 3$, $\alpha = 0.6$, $\tau = 0.5$), and all VLM model versions.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: All main result tables report video-level bootstrap 95% confidence intervals. Section 4.7 reports paired bootstrap comparisons with 95% CIs for Δ Human Pass@2, explicitly treating estimates with CIs crossing zero as inconclusive. The bootstrap is computed over videos with standard resampling.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.

- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The break-even analysis in Section 5 reports per-video diagnosis cost ($0.24) and per-video re-rendering cost ($0.52). Feature extraction uses a single consumer GPU; CPU-only recomputation from serialized predictions completes in seconds. VLM judging uses hosted GPT-4V endpoints.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: Section 7 addresses ethical considerations including misuse risk (misleading media), provenance tracking, watermarking, and human review. The annotation protocol was IRB-approved. No human-subject data or PII was collected. All source clips are from permissively licensed public datasets.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Section 7 discusses positive impact (improved controllability and quality of generated video) and negative impact (potential misuse to strengthen misleading media). Mitigation strategies are suggested: provenance tracking, watermarking, human review, and access control in sensitive deployments.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [Yes]

    Justification: VideoGenDoctor is a diagnostic tool, not a generative model—it does not release any image/video generator that could be directly misused. The released benchmark videos are perturbed diagnostic clips, not photorealistic deepfakes. The paper recommends pairing automated repair with provenance tracking, watermarking, and human review in sensitive deployments (Section 7). The failure taxonomy exposes configurable thresholds for human override.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: All existing assets (CogVideoX, Wan, Stable Video Diffusion, HunyuanVideo, OpenCLIP, RAFT, Farneback, InsightFace, YOLOv8, GPT-4V) are cited with version identifiers. The 18 clean source clips are from publicly available video datasets under permissive licences; sources are listed in the construction manifest (Appendix B).

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.

- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: VideoGenDoctor-Bench-v0 (324 controlled videos with 2,016 evidence spans, plus 240 real-failure videos) and the reference implementation will be released on GitHub under an MIT license. Documentation includes the taxonomy schema (Appendix C), patch compiler specification (Appendix E), annotation guide (Appendix F), VLM baseline prompt templates (Appendix H), and a comprehensive README.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [Yes]

    Justification: The annotation protocol involved human annotators from the authors' institution. Appendix F summarizes the annotation guide, and the annotation interface is shown in Figure 14. Annotators reviewed perturbed videos with specifications and marked failure codes, evidence spans, and keyframes. A dual-annotated reliability subset (120 videos, 768 spans) is reported. Three-annotator stability analysis is in Appendix B.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [Yes]

Justification: The annotation protocol was reviewed and approved by the authors' institutional review board, as stated in the acknowledgments. The annotation task involved reviewing publicly available video clips against textual specifications; participants faced no risks beyond those of ordinary screen-based work.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: VLM-based judges (GPT-4V, Qwen2-VL) are used as baseline comparison methods and as an optional verification stage, not as an important, original, or non-standard component of the core VideoGenDoctor method. The Stage-1 rule engine and patch compiler operate independently of LLMs. LLMs were not used to draft the paper's core scientific argument.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.